课程编号：　　课程性质：必修

卫星导航算法与程序设计 1

成果总结报告

专业：导航工程

姓名：SZQ

2026 年 3 月 2 日　至　2026 年 5 月 5 日

目录

1 程序设计概述 1
1.1 课程目的 1
1.2 软件介绍 1
1.2.1 总体设计 1
1.2.2 模块设计 1
1.2.3 编程语言 1
1.2.4 软件功能 2
1.3 完成情况 2
2 算法与程序设计 4
2.1 时间与坐标转换 4
2.1.1 时间转换 4
2.1.2 坐标转换 7
2.2 NovAtel 数据获取 11
2.2.1 数据格式 11
2.2.2 解码函数 12
2.2.3 字节提取 14
2.2.4 观测数据提取 14
2.3 卫星位置计算 19
2.3.1 GPS 卫星 19
2.3.2 BDS 卫星 25
2.3.3 星历检查 30
2.4 误差改正 31
2.4.1 大气误差 31
2.4.2 粗差探测 34
2.5 单点定位与测速 36
2.5.1 信号发射时刻卫星的位置 36
2.5.2 单点定位 38
2.5.3 单点测速 42
2.6 获取实时数据流 44
2.7 主函数设计 45
3 结果精度分析 48

1 程序设计概述

1.1 课程目的

基于 NovAtel OEM719 GNSS 接收机板卡，开发卫星导航定位软件，包括 GPS/BDS 单点定位与 RTK 相对定位，理解并掌握卫星导航定位算法及其程序的设计方法与实现过程。

1.2 软件介绍

1.2.1 总体设计

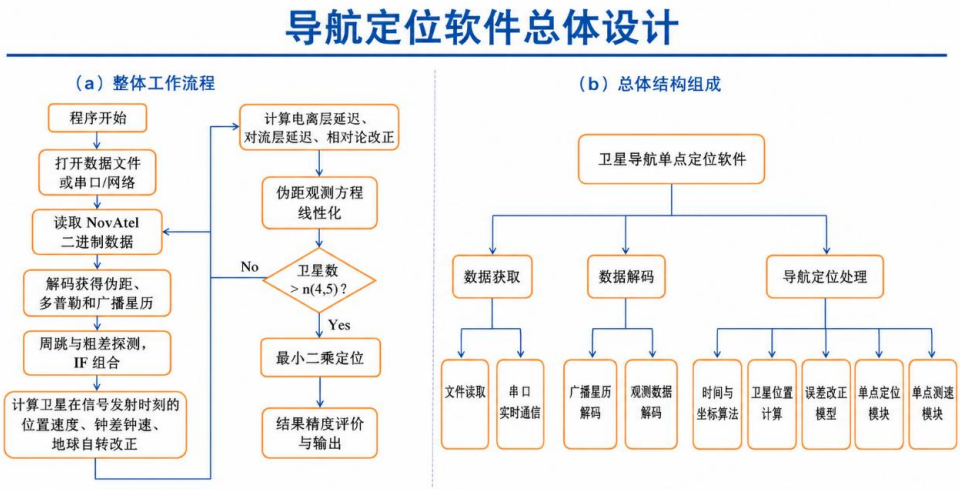


图 1 导航定位软件总体设计图

1.2.2 模块设计

针对每个模块，本软件均设计了相对独立的、具有一定功能的子函数。

1.2.3 编程语言

1.2.3.1 开发环境

本软件在 Microsoft Visual Studio 2022 集成开发环境中，使用 C++语言进行开发。

1.2.3.2 编程规范

为保证代码的专业性、可读性和可维护性，整个项目严格遵守了课程中强调的 C++编码与命名等 规则，具体规则如下：

（1） 命名规则：函数名采用大驼峰命名法；

（2） 变量规则：多个变量具有内部关联性时，构造相应的结构体或者向量来统一进行存储；

（3） 注释规则：对于关键函数、参数、结构体等，均有相应的详细的注释。

1.2.4 软件功能

为实现导航定位软件的数据读取、数据解码、卫星位置计算、误差改正、单点定位与单点测速等功能，本文设计并实现了一套基于 NovAtel OEM7 原始二进制数据的 GNSS 导航定位处理程序，其主要功能与技术规格如下。

1.2.4.1 输入

（1） GNSS 系统与信号：软件支持 GPS 与 BDS 双系统双频观测数据处理，其中 GPS 主要采用 L1/L2 信号，BDS 主要采用 B1I/B3I 信号。

（2） 数据内容：软件能够解析和计算的数据为 GPS/BDS 广播星历和观测文件

（3） 数据获取模式：软件支持文件模式和实时模式两种数据获取模式

1.2.4.2 处理功能

（1） 对原始 OEM7 数据进行解码，提取伪距、载波相位、多普勒、信噪比等观测值

（2） 提供时间系统与坐标系统转换，实现历元时间、发射时刻、ECEF 与大地坐标转换

（3） 计算卫星在信号发射时刻的位置、速度与钟差，并进行星历有效性检查

（4） 对观测数据进行粗差剔除与对流层延迟改正，保证定位观测质量

（5） 基于迭代最小二乘方法完成单点定位与单点测速解算，并输出 PDOP 与中误差

（6） 提供矩阵运算和站心坐标转换支持，为定位解算和速度计算提供基础支撑

1.2.4.3 输出

输出接收机三维坐标、速度、大地经纬高、接收机钟差、PDOP 及参与解算卫星数量，并可保存中间计算结果用于分析与调试。

1.3 完成情况

打开 RTKNav.sln 文件，在 GPS_Structs.h 文件中选择 1（文件模式）或者 0（实时模式）。

1.3.1 文件模式

将 FILEMODE 选择 1 并进行编译，再运行可执行程序。控制台显示如图 2 所示，输出结果至 oem.log 文件中，结果如图 3 所示。

```
开始处理...
SPP: 2409 273995.000 X:-2267335.1970 Y:5008648.3409 Z:3222378.2018 Vx:0.0102 Vy:-0.0340 Vz:-0.0467 B: 30.54299707 L:114.
35550412 H:  34.294 GPS Clk:   0.000ns BDS Clk:  25.891ns PDOP:   1.420 Sigma:   2.252 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 273996.000 X:-2267335.2093 Y:5008648.3623 Z:3222378.2091 Vx:-0.0037 Vy:0.0122 Vz:0.0057 B: 30.54299702 L:114.3
5550414 H:  34.319 GPS Clk:   0.000ns BDS Clk:  25.978ns PDOP:   1.420 Sigma:   2.251 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 273997.000 X:-2267335.2294 Y:5008648.3972 Z:3222378.2235 Vx:-0.0013 Vy:-0.0040 Vz:0.0116 B: 30.54299695 L:114.
35550418 H:  34.360 GPS Clk:   0.000ns BDS Clk:  26.095ns PDOP:   1.420 Sigma:   2.256 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 273998.000 X:-2267335.2416 Y:5008648.4166 Z:3222378.2239 Vx:-0.0230 Vy:0.0161 Vz:-0.0055 B: 30.54299684 L:114.
35550421 H:  34.380 GPS Clk:   0.000ns BDS Clk:  26.206ns PDOP:   1.420 Sigma:   2.257 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 273999.000 X:-2267335.2529 Y:5008648.4347 Z:3222378.2128 Vx:-0.0091 Vy:0.0031 Vz:-0.0057 B: 30.54299666 L:114.
35550424 H:  34.393 GPS Clk:   0.000ns BDS Clk:  26.175ns PDOP:   1.421 Sigma:   2.259 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 274000.000 X:-2267335.2617 Y:5008648.4589 Z:3222378.2147 Vx:-0.0075 Vy:-0.0075 Vz:0.0006 B: 30.54299656 L:114.
35550422 H:  34.416 GPS Clk:   0.000ns BDS Clk:  26.056ns PDOP:   1.421 Sigma:   2.263 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 274001.000 X:-2267335.2733 Y:5008648.4757 Z:3222378.1998 Vx:0.0056 Vy:-0.0137 Vz:0.0025 B: 30.54299635 L:114.3
5550426 H:  34.426 GPS Clk:   0.000ns BDS Clk:  25.905ns PDOP:   1.421 Sigma:   2.264 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 274002.000 X:-2267335.2943 Y:5008648.5074 Z:3222378.1894 Vx:-0.0157 Vy:0.0270 Vz:0.0167 B: 30.54299610 L:114.3
5550432 H:  34.453 GPS Clk:   0.000ns BDS Clk:  25.726ns PDOP:   1.421 Sigma:   2.266 GPSSats:  0 BDSSats: 17 Sats: 26

SPP: 2409 274003.000 X:-2267335.4005 Y:5008649.6396 Z:3222377.9624 Vx:-0.0008 Vy:0.0024 Vz:0.0099 B: 30.54298941 L:114.3
5550047 H:  35.263 GPS Clk:   2.251ns BDS Clk:  27.903ns PDOP:   1.125 Sigma:   1.907 GPSSats:  9 BDSSats: 17 Sats: 26

SPP: 2409 274004.000 X:-2267335.4111 Y:5008649.6418 Z:3222377.9513 Vx:0.0028 Vy:-0.0539 Vz:-0.0147 B: 30.54298929 L:114.
35550056 H:  35.263 GPS Clk:   1.879ns BDS Clk:  27.587ns PDOP:   1.125 Sigma:   1.904 GPSSats:  9 BDSSats: 17 Sats: 26
```

图 2 文件模式控制台输出内容

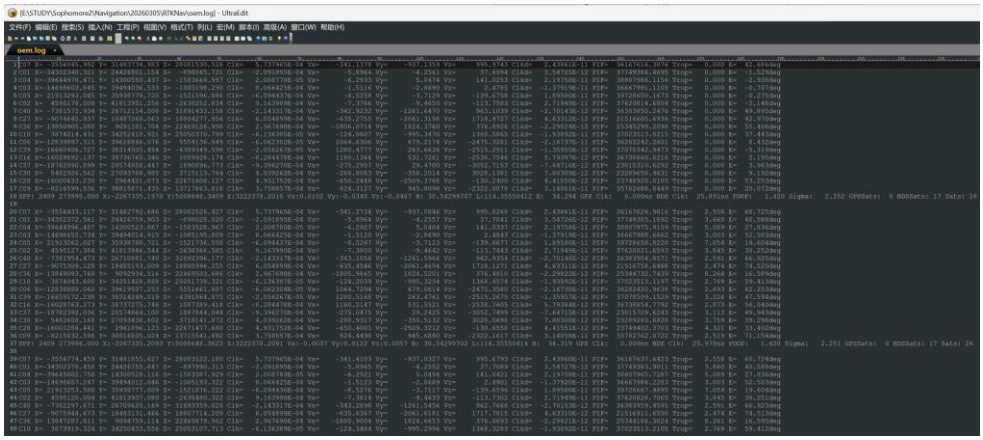


图 3 文件模式 oem.log 结果文件中输出内容

1.3.2 实时模式

将 FILEMODE 选择 0 并进行编译，再运行可执行程序。控制台显示如图 4 所示。

```
开始处理...
SPP: 2422 13969.000 FAILED

SPP: 2422 13970.000 FAILED

SPP: 2422 13971.000 FAILED

SPP: 2422 13972.000 FAILED

SPP: 2422  13973.000 X:-2267336.2481 Y:5008651.4527 Z:3222377.0427 Vx:0.0122 Vy:-0.0499 Vz:-0.0312 B: 30.54297309 L:114.
35550072 H:  36.519 GPS Clk:  14.702ns BDS Clk:   0.000ns PDOP:   2.255 Sigma:   1.204 GPSSats:  8 BDSSats:  0 Sats: 23

SPP: 2422  13974.000 X:-2267336.2247 Y:5008651.4100 Z:3222377.0160 Vx:-0.0160 Vy:0.0072 Vz:0.0011 B: 30.54297310 L:114.3
5550068 H:  36.464 GPS Clk:  14.402ns BDS Clk:   0.000ns PDOP:   2.254 Sigma:   1.198 GPSSats:  8 BDSSats:  0 Sats: 23

SPP: 2422  13975.000 X:-2267335.4840 Y:5008648.9881 Z:3222375.2776 Vx:-0.0174 Vy:0.0408 Vz:0.0366 B: 30.54297111 L:114.3
5550406 H:  33.417 GPS Clk:   7.545ns BDS Clk:  24.055ns PDOP:   1.203 Sigma:   1.204 GPSSats:  8 BDSSats: 15 Sats: 23

SPP: 2422  13976.000 X:-2267335.4424 Y:5008648.9043 Z:3222375.2473 Vx:0.0165 Vy:-0.0064 Vz:-0.0116 B: 30.54297130 L:114.
35550403 H:  33.321 GPS Clk:   7.148ns BDS Clk:  23.587ns PDOP:   1.203 Sigma:   1.206 GPSSats:  8 BDSSats: 15 Sats: 23

SPP: 2422  13977.000 X:-2267335.4552 Y:5008648.9173 Z:3222375.2619 Vx:-0.0157 Vy:0.0093 Vz:-0.0288 B: 30.54297134 L:114.
35550409 H:  33.344 GPS Clk:   7.073ns BDS Clk:  23.504ns PDOP:   1.203 Sigma:   1.216 GPSSats:  8 BDSSats: 15 Sats: 23

SPP: 2422  13978.000 X:-2267335.4477 Y:5008648.8904 Z:3222375.2731 Vx:0.0109 Vy:-0.0299 Vz:-0.0115 B: 30.54297155 L:114.
35550414 H:  33.325 GPS Clk:   6.860ns BDS Clk:  23.244ns PDOP:   1.203 Sigma:   1.215 GPSSats:  8 BDSSats: 15 Sats: 23

SPP: 2422  13979.000 X:-2267335.4686 Y:5008648.9202 Z:3222375.3012 Vx:-0.0054 Vy:0.0064 Vz:0.0057 B: 30.54297161 L:114.3
5550420 H:  33.371 GPS Clk:   6.724ns BDS Clk:  23.149ns PDOP:   1.203 Sigma:   1.222 GPSSats:  8 BDSSats: 15 Sats: 23
```

图 4 实时模式控制台输出内容

2 算法与程序设计

2.1 时间与坐标转换

2.1.1 时间转换

2.1.1.1 整体思路

时间转换的整体思路如图 5 所示。

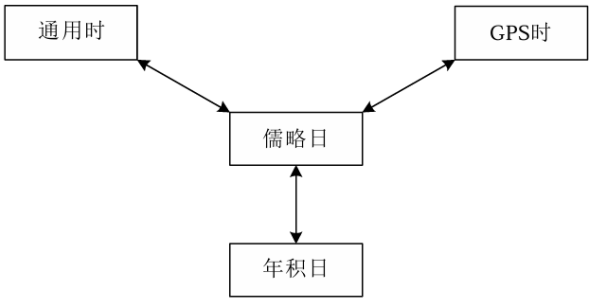


图 5 时间转换的整体思路

2.1.1.2 结构体定义

（1） 通用时

```
struct COMMONTIME {
    short Year;
    unsigned short Month;
    unsigned short Day;
    unsigned short Hour;
    unsigned short Minute;
    double Second;
    COMMONTIME()
    {
        Year = 0;
        Month = 0;
        Day = 0;
        Hour = 0;
        Minute = 0;
        Second = 0.0;
    }
};
```

（2） 儒略时

```
struct MJDTIME {
```

```
    int Days;
    double FracDay;
    MJDTIME() {
        Days = 0;
        FracDay = 0.0;
    }
};
```

（3） GPS 时

```
struct GPSTIME {
    short Week;
    double SecOfWeek;

    GPSTIME() {
        Week = 0;
        SecOfWeek = 0.0;
    }
    GPSTIME(short w,double sow):Week(w),SecOfWeek(sow){}
};
```

2.1.1.3 通用时转化为简化儒略日

（1） 计算公式

$$JD = \mathrm{INT}[365.25y] + \mathrm{INT}[30.6001(m+1)] + D + \frac{UT}{24} + 1720981.5 \tag{1}$$

$$MJD = JD - 2400000.5 \tag{2}$$

其中，若$M \le 2$，则$y = Y - 1$，$m = M + 12$；否则$y = Y$，$m = M$。

（2） 程序实现

```
void CommonTimeToMJDTime(const COMMONTIME* ct, MJDTIME* mjd) {
    double y, m;
    if (ct->Month <= 2) {
        y = ct->Year - 1;
        m = ct->Month + 12;
    }
    else {
        y = ct->Year;
        m = ct->Month;
    }
    mjd->Days = int(365.25 * y) + int(30.6001 * (m + 1)) + ct->Day
        + 1720981.5 - 2400000.5;
    // 设计两个变量是为了保证精度，不能直接用两个变量再分开，这样的话 double 不能完全保证精度
    double ut = ct->Hour + ct->Minute / 60.0 + ct->Second / 3600.0;
```

```
    mjd->FracDay = ut / 24;
}
```

2.1.1.4 简化儒略日转化为通用时

（1） 计算公式

$$a = INT(JD + 0.5) \tag{3}$$

$$b = a + 1537 \tag{4}$$

$$c = INT(\frac{b - 122.1}{365.25}) \tag{5}$$

$$d = INT(365.25 \times c) \tag{6}$$

$$e = INT(\frac{b - d}{30.6001}) \tag{7}$$

则有，

$$Day = b - d - INT(30.6001 \times e) + FRAC(JD + 0.5) \tag{8}$$

$$Month = e - 1 - 12 \times INT(\frac{e}{14}) \tag{9}$$

$$Year = c - 4715 - INT(\frac{7 + Month}{10}) \tag{10}$$

$$Week = mod(INT(JD + 0.5), 7) \tag{11}$$

（2） 程序实现

```
void MJDTimeToCommonTime(const MJDTIME* mjd, COMMONTIME* ct) {
    int a, b, c, d, e;
    a = int(mjd->Days + 2400000.5 + 0.5);
    b = a + 1537;
    c = int((b - 122.1) / 365.25);
    d = int(365.25 * c);
    e = int((b - d) / 30.6001);
    ct->Day = b - d - int(30.6001 * e);
    ct->Month = e - 1 - 12 * int(e / 14);
    ct->Year = c - 4715 - int((7 + ct->Month) / 10);
    ct->Hour = int(mjd->FracDay * 24.0);
    ct->Minute = int((mjd->FracDay * 24.0 - ct->Hour) * 60);
    ct->Second = ((mjd->FracDay * 24.0 - ct->Hour) * 60 - ct->Minute) * 60;
}
```

2.1.1.5 GPS 时转化为简化儒略日

（1） 计算公式

$$MJD = 44244 + GPSWeek \times 7 + \frac{SecOfWeek}{86400} \tag{12}$$

（2） 程序实现

```
1.  void GPSTimeToMJDTime(const GPSTIME* gt, MJDTIME* mjd) {
2.      double day = gt->SecOfWeek / 86400.0;
3.      mjd->Days = 44244 + gt->Week * 7 + int(day);
4.      mjd->FracDay = day - int(day);
5.  }
```

2.1.1.6 简化儒略日转化为 GPS 时

（1） 计算公式

$$GPSWeek = INT\left(\frac{MJD - 44244}{7}\right) \tag{13}$$

$$SecOfWeek = (MJD - 44244 - GPSWeek \times 7) \times 86400 \tag{14}$$

（2） 程序实现

```
1.  void MJDTimeToGPSTime(const MJDTIME* mjd, GPSTIME* gt) {
2.      gt->Week = int((mjd->Days - 44244) / 7.0);
3.      gt->SecOfWeek = (mjd->Days + mjd->FracDay - 44244 - gt->Week * 7) * 86400;
4.  }
```

2.1.1.7 通用时与 GPS 时互相转换

（1） 计算公式

借助中间变量 MJD，来实现通用时与 GPS 时的转化。

（2） 程序实现

```
1.  void CommonTimeToGPSTime(const COMMONTIME* ct, GPSTIME* gt) {
2.      MJDTIME mjd;
3.      CommonTimeToMJDTime(ct, &mjd);
4.      MJDTimeToGPSTime(&mjd, gt);
5.  }
6.  void GPSTimeToCommonTime(const GPSTIME* gt, COMMONTIME* ct) {
7.      MJDTIME mjd;
8.      GPSTimeToMJDTime(gt, &mjd);
9.      MJDTimeToCommonTime(&mjd, ct);
10. }
```

2.1.2 坐标转换

2.1.2.1 结构体定义

（1） 笛卡尔坐标系

```
union XYZ {
    double xyz[3];
    struct {
        double x;
        double y;
        double z;
    };
};
```

（2） 大地坐标系

```
union BLH {
    double blh[3];
    struct {
        double b;
        double l;
        double h;
    };
};
```

（3） 测站地平坐标系

```
union ENU {double enu[3];
    struct {
        double e;
        double n;
        double u;
    };
};
```

2.1.2.2 大地坐标系转换为笛卡尔坐标系

（1） 计算公式

$$\mathrm{X} = (\mathrm{N} + \mathrm{H})\cos \mathrm{B}\cos \mathrm{L} \tag{15}$$

$$Y = (N + H)cosBsinL \tag{16}$$

$$Z = [N(1 - e^2) + H]sinB \tag{17}$$

$$Z = \left[N\frac{b^2}{a^2} + H\right]sinB \tag{18}$$

其中，N为卯酉圈的半径，满足$N = \frac{a}{\sqrt{1-e^2 sin^2 B}}$，$e$为第一偏心率，满足$e^2 = \frac{a^2-b^2}{a^2} = 2f - f^2$。

（2） 程序实现

```
void BLHToXYZ(const double blh[3], double  xyz[3], const double R, const double F) {
    double e2 = 2 * F - F * F;
    double n = R / sqrt(1 - e2 * SQR(sin(blh[0])));
    xyz[0] = (n + blh[2]) * cos(blh[0]) * cos(blh[1]);
    xyz[1] = (n + blh[2]) * cos(blh[0]) * sin(blh[1]);
    xyz[2] = (n * (1 - e2) + blh[2]) * sin(blh[0]);
};
```

2.1.2.3 笛卡尔坐标系转换为大地坐标系

（1） 计算公式

$$L = arctan\left(\frac{Y}{X}\right) \tag{19}$$

$$B = arctan\left(\frac{Z + \Delta Z}{\sqrt{X^2 + Y^2}}\right) \tag{20}$$

$$H = \sqrt{X^2 + Y^2 + (Z + \Delta Z)^2} - N \tag{21}$$

其中，$sinB = \frac{Z+\Delta Z}{\sqrt{X^2+Y^2+(Z+\Delta Z)^2}}$，$N = \frac{a}{\sqrt{1-e^2 sin^2 B}}$，$\Delta Z = Ne^2 sinB$，需要迭代计算，初始时，令$\Delta Z = e^2 Z$。

（2） 程序实现

```
void XYZToBLH(const double xyz[3], double blh[3], const double R, const double F) {
    double e2 = 2.0 * F - F * F;
    double p = sqrt(SQR(xyz[0]) + SQR(xyz[1]));
    blh[1] = atan2(xyz[1], xyz[0]);
    if (p < 1E-12) {
        blh[0] = (xyz[2] >= 0.0 ? PI / 2.0 : -PI / 2.0);
        blh[2] = fabs(xyz[2]) - R * sqrt(1.0 - e2);
        return;
    }
    double delta_z = e2 * xyz[2] , dz = 0.0;
    double sinB = 0.0;
    double n = 0.0;
    blh[0] = atan((xyz[2] + delta_z)/ p);
    while (true) {
        dz = delta_z;
        blh[0] = atan((xyz[2] + delta_z) / p);
        sinB = sin(blh[0]);
        n = R / sqrt(1.0 - e2 * SQR(sinB));
        delta_z = n * e2 * sinB;
        if (fabs(delta_z - dz) < 1E-12) break;
    }
    blh[0] = atan2(xyz[2] + delta_z, p);
    n = R / sqrt(1.0 - e2 * SQR(sin(blh[0])));
```

```
    blh[2] = sqrt(SQR(xyz[0]) + SQR(xyz[1]) + SQR(xyz[2] + delta_z)) - n;
}
```

2.1.2.4 测站地平定位误差计算

（1） 计算公式

已知测站的大地坐标为(B, L, H)，转换矩阵为：

$$R = \begin{bmatrix} -\sin L & \cos L & 0 \\ -\sin B \cos L & -\sin B \sin L & \cos B \\ \cos B \cos L & \cos B \sin L & \sin B \end{bmatrix} \tag{22}$$

测站地平系的定位误差（X_0为测站的精确坐标）：

$$[dE, dN, dH]^T = R[x - x_0, y - y_0, z - z_0]^T \tag{23}$$

（2） 程序实现

```
void CompEnudPos(const double X0[], const double Xr[], double dENU[], const double R, const double F) {
    double blh[3];
    XYZToBLH(X0, blh, R, F);
    Vector3d dxyz;
    dxyz <<
        Xr[0] - X0[0],
        Xr[1] - X0[1],
        Xr[2] - X0[2];
    Vector3d denu = BLH2ENUMat(blh) * dxyz;
    dENU[0] = denu[0];
    dENU[1] = denu[1];
    dENU[2] = denu[2];
}
```

2.1.2.5 卫星高度角与方位角计算

（1） 计算公式

卫星高度角和方位角计算（X_0为测站坐标，X^s为卫星坐标）：

$$[dE, dN, dH]^T = R[x^s - x_0, y^s - y_0, z^s - z_0]^T \tag{24}$$

高度角：

$$Elev = atan\left(dH / \sqrt{dN^2 + dE^2}\right) \tag{25}$$

方位角：

$$Azim = atan2(dE, dN) \tag{26}$$

（2） 程序实现

```
void CompSatElAz(const double Xr[], const double Xs[], double R,double F,double* Elev, double* Azim) {
    Vector3d dx;
```

```
        dx <<
            Xr[0] - Xs[0],
            Xr[1] - Xs[1],
            Xr[2] - Xs[2];
        double blh[3], enu[3];
        XYZToBLH(Xr, blh, R, F);
        Vector3d de = BLH2ENUMat(blh) * ( - dx);
        enu[0] = de[0];
        enu[1] = de[1];
        enu[2] = de[2;
        *Elev = atan(enu[2]/ sqrt(SQR(enu[1])+SQR(enu[0])));
        *Azim = atan2(enu[0], enu[1]);
        if (*Azim < 0.0) {
            *Azim += 2.0 * PI;
        }
    }
```

2.2 NovAtel 数据获取

2.2.1 整体流程

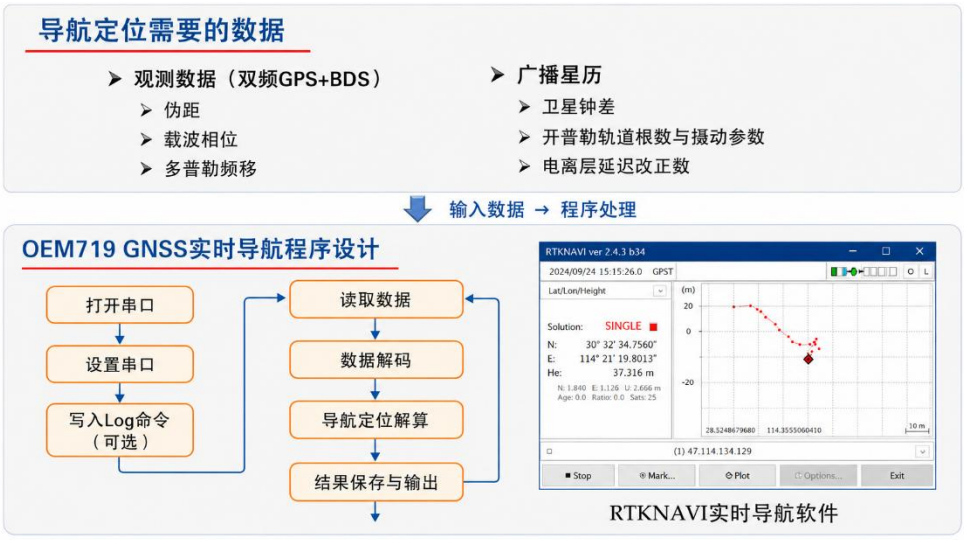


图 6 NovAtel 数据获取整体流程示意图

2.2.2 数据格式

本项目使用的 GNSS 数据由 NovAtel 公司的 OEM7 接收机生成，采用二进制格式记录，可以通过串口或数据文件读取。每条消息由以下三部分组成：

2.2.2.1 数据帧结构

表 1 数据帧结构说明

部分	说明
Header	固定 28 字节，包含同步字、消息类型、时间信息等。其中前 3 字节为同步标志 0xAA 0x44 0x12，用于识别一帧数据的起始
Data	可变长度，存储实际 GNSS 数据，如伪距、载波相位、多普勒观测值及星历信息
CRC	固定 4 字节，32 位 CRC 校验码，用于验证 Header 与 Data 的完整性

2.2.2.2 主要消息类型

表 2 主要消息类型及其说明

Message ID	说明
43	观测值
7	GPS 广播星历
1696	BDS 广播星历
42	定位结果

2.2.2.3 CRC 校验

（1） 说明

每帧数据尾部的 CRC 用于校验 Header 与 Data 内容是否正确，保证数据传输可靠性。本项目在 Decode.cpp 中实现了 CRC 校验函数，通过循环异或计算每帧数据的 CRC 码。

（2） 程序实现

```
#define POLYCRC32		0xEDB88320u
unsigned int crc32(const unsigned char* buff, int len) {
    int i, j;
    unsigned int crc = 0;
    for (i = 0; i < len; i++) {
        crc ^= buff[i];
        for (j = 0; j < 8; j++){
            if (crc & 1) crc = (crc >> 1) ^ POLYCRC32;
            else crc >>= 1;
        }
    }
    return crc;
}
```

2.2.3 解码函数

解码函数用于从读取到的缓冲区（Buff）中解析各类 GNSS 消息，生成可用的观测值、星历和定位结果数据。

2.2.3.1 解码主函数

（1） 功能

循环解析 Buff 缓冲区中所有消息，区分消息类型，提取观测数据、GPS/BDS 星历及定位结果。

（2） 处理逻辑

①查找消息同步字符 AA 44 12；

②提取消息头长度（28 字节）和消息长度；

③进行 CRC 检验，确保数据完整；

④根据 MsgID 调用对应解码子函数，如观测数据（decode_rangeb_oem7）、GPS 星历（decode_gpsephem）、BDS 星历（decode_bdsephem）；

⑤处理剩余字节，确保缓冲区中不足一帧的数据拼接到下一次读取。

（3） 程序实现

```
1.  int DecodeNovOem7Dat(unsigned char buff[], int& len, EPOCHOBSDATA* obs, GPSEPH geph[], GPSEPH beph[]) {
2.      uint16_t MsgID = 0 , MsgLen = 0;
3.      unsigned char Msg[10240];
4.      int Type = 0 , i = 0;
5.    while (true) {
6.      for (; i < len; i++) {
7.        if (buff[i] == 0xAA && buff[i + 1] == 0x44 && buff[i + 2] == 0x12) break;
8.      }
9.      if ((i + 28) > len)    break;
10.     memcpy(Msg, buff + i, 28);
11.     MsgID = US2(Msg+ 4);
12.     MsgLen = US2(Msg + 8);
13.     if ((i + 28 + MsgLen + 4) > len) break;
14.     memcpy(Msg + 28, buff + i + 28, MsgLen + 4);
15.     if (crc32(Msg, 28 + MsgLen) != UI4(Msg + 28 + MsgLen)) {
16.       i = i + 3;
17.       continue;
18.     }
19.     switch (MsgID)   {
20.     case 43:
21.       Type = decode_rangeb_oem7(buff + i, obs);
22.       break;
23.     case 7:
24.       Type = decode_gpsephem(buff + i, geph);
25.       break;
26.     case 1696:
```

```
            Type = decode_bdsephem(buff + i, beph);
            break;
        default:
            break;
        }
        i = i + 28 + MsgLen + 4;
        if (MsgID == 43 && Type == 1) break;
        if (FILEMODE == 1 && Type == 1) break;
    }
    memcpy(buff, buff + i, len - i);
    len = len - i;
    return Type;
}
```

2.2.4 字节提取

表 3 字节提取函数汇总表

名称	说明
D8	提取 8 字节 double
F4	提取 4 字节 float
I4	提取 4 字节 int（有符号整数）
UI4	提取 4 字节 unsigned int（无符号整数）
I2	提取 2 字节 int
UI2	提取 2 字节 unsigned int

以 D8 函数为例，其程序实现为：

```
double D8(unsigned char* p){
    double r;
    memcpy(&r, p, 8);
    return r;
}
```

2.2.5 观测数据提取

2.2.5.1 观测值

（1） 结构体定义

```
//每个历元的观测值
struct EPOCHOBSDATA {
    GPSTIME         Time;       //（接收机钟的表面时）
    short       SatNum;
    SATOBSDATA      SatObs[MAXCHANNUM];     //原始观测
    SATMIDRES       SatPVT[MAXCHANNUM];     //卫星状态
    MWGF            CombObs[MAXCHANNUM];    //组合观测
    EPOCHOBSDATA(){
```

```
        SatNum = 0;
    }
};
// 每颗卫星的观测值
struct SATOBSDATA {
    short Prn;
    GNSSSYS System;
    double P[2], L[2], D[2];    //双频
    double  cn0[2], LockTime[2];
    unsigned char half[2];
    bool valid;
    SATOBSDATA(){
        Prn = 0;
        System = UNKNOWN;
        for (int i = 0; i < 2; i++)
            P[i] = L[i] = D[i] = 0.0;
        valid = false;
    }
};
```

（2） 功能

解码一条完整 Rangeb 消息，解析当前历元的所有卫星信号观测值。

（3） 处理逻辑

①解码观测时刻，用 GPSTIME 结构体表示；

②获取所有卫星的观测值数量；

③循环解析各卫星各信号：读取跟踪状态，判断卫星系统和信号类型，并填充到对应的 SATOBSDATA 结构体数组；

④统计有效卫星数量。

（4） 程序实现

```
int decode_rangeb_oem7(unsigned char* buff, EPOCHOBSDATA* obs) {
    int ObsNum, i = 0, Freq, prn, m = 0, count = 0;
    double wl;
    unsigned char* p = buff + 32;
    unsigned int ch_tr_status;
    int PhaseLockFlag, CodeLockedFlag, ParityFlag, SatSystem, SignalType;
    GNSSSYS sys;
    bool FindFlag;
    obs->Time.Week = US2(buff + 14);
```

```
10.     obs->Time.SecOfWeek = UI4(buff + 16) / 1000.0;
11.     ObsNum = UI4(buff + 28);
12.     obs->SatNum = 0;
13.     memset(obs->SatObs, 0, MAXCHANNUM * sizeof(SATOBSDATA));
14.     for (i = 0/*, p += 4*/; i < ObsNum; i++,p+=44) {
15.         ch_tr_status = UI4(p + 40);
16.         PhaseLockFlag = (ch_tr_status >> 10) & 0x01;     //相位跟踪锁定
17.         ParityFlag = (ch_tr_status >> 11) & 0x01;          //半周状态
18.         CodeLockedFlag = (ch_tr_status >> 12) & 0x01;   //伪距相位观测值的跟踪状态
19.         SatSystem = (ch_tr_status >> 16) & 0x07;           //卫星类型
20.         SignalType = (ch_tr_status >> 21) & 0x1F;          //信号类型
21.         prn = US2(p);
22.         if (SatSystem != 4 && SatSystem != 0) continue;
23.         if (SatSystem == 0) {
24.             sys = GPS;
25.             if (SignalType == 0) {
26.                 Freq = 0;
27.                 wl = WL1_GPS;
28.             }
29.             else if (SignalType == 9) {
30.                 Freq = 1;
31.                 wl = WL2_GPS;
32.             }
33.             else continue;
34.         }
35.         else if (SatSystem == 4) {
36.             sys = BDS;
37.             if (SignalType == 0 || SignalType == 4) {
38.                 Freq = 0;
39.                 wl = WL1_BDS;
40.             }
41.             else if (SignalType == 2 || SignalType == 6) {
42.                 Freq = 1;
43.                 wl = WL3_BDS;
44.             }
45.             else {
46.                 continue;
47.             }
48.         }
49.         else continue;
50.         FindFlag = false;
51.         for (int j = 0; j < MAXCHANNUM; j++) {
```

```
            if (obs->SatObs[j].System == sys && obs->SatObs[j].Prn == prn) {
                m = j;
                FindFlag = true;
                break;
            }
            if (obs->SatObs[j].Prn == 0) {
                m = j;
                FindFlag = true;
                break;
            }
        }
        if (FindFlag == false) continue;
        obs->SatObs[m].Prn = prn;
        obs->SatObs[m].System = sys;
        obs->SatObs[m].P[Freq] = (CodeLockedFlag == 1 ? D8(p + 4) : 0.0);
        obs->SatObs[m].L[Freq] = -wl * (PhaseLockFlag == 1 ? D8(p + 16) : 0.0);
        obs->SatObs[m].D[Freq] = -wl * F4(p + 28);
        obs->SatObs[m].LockTime[Freq] = F4(p + 36);
        obs->SatObs[m].cn0[Freq] = F4(p + 32);
        obs->SatObs[m].half[Freq] = ParityFlag;
    }
    for (i = 0; i < MAXCHANNUM; i++) {
        if (obs->SatObs[i].System != UNKNOWN && obs->SatObs[i].Prn != 0) {
            obs->SatNum++;
        }
    }
    return 1;
}
```

2.2.5.2 GPS/BDS 广播星历

（1） 结构体定义

```
struct GPSEPH {
    short PRN;
    GNSSSYS System;
    GPSTIME TOC, TOE;      //钟差和星历拟合的参考时间
    double a0, a1, a2;     //钟差，钟漂，频漂
    double IODE, IODC;
    double sqrt_A, M0, e, OMEGA, i0, omega;
    double Crc, Crs, Cuc, Cus, Cic, Cis;
    double deltaN, OMEGA_dot, IDOT;
```

```
    int SVhealth;
    double tgd1, tgd2;
};
```

（2） 功能

解析单颗卫星的广播星历数据，并存入星历数组。

（3） 处理逻辑

①获取卫星 PRN 号，判断是否在数组范围内；

②按 OEM 星历格式解析各字段，包括参考时刻、钟差、轨道根数、摄动参数等；

③BDS 星历参考时间 TOC/TOE 使用 GPSTIME 结构体表示。

（4） 程序实现

```
int decode_gpsephem(unsigned char* buff, GPSEPH* geph) {
    int prn;
    unsigned char* p = buff + 28;
    prn = UI4(p);
    if (prn<1 || prn>MAXGPSNUM) return 0;
    GPSEPH* eph = geph + prn - 1;
    eph->PRN = prn;
    eph->System = GPS;
    eph->TOC.Week = eph->TOE.Week = UI4(p + 24);
    eph->TOE.SecOfWeek = D8(p + 32);
    eph->TOC.SecOfWeek = D8(p + 164);
    eph->a0 = D8(p + 180);
    eph->a1 = D8(p + 188);
    eph->a2 = D8(p + 196);
    eph->IODE = UI4(p + 16);
    eph->IODC = UI4(p + 160);
    eph->sqrt_A = sqrt(D8(p + 40));
    eph->M0 = D8(p + 56);
    eph->e = D8(p + 64);
    eph->OMEGA = D8(p + 144);
    eph->i0 = D8(p + 128);
    eph->omega = D8(p + 72);
    eph->Cic = D8(p + 112);
    eph->Cis = D8(p + 120);
    eph->Crc = D8(p + 96);
    eph->Crs = D8(p + 104);
    eph->Cuc = D8(p + 80);
    eph->Cus = D8(p + 88);
    eph->deltaN = D8(p + 48);
    eph->OMEGA_dot = D8(p + 152);
    eph->SVhealth = UI4(p + 12);
    eph->IDOT = D8(p + 136);
    eph->tgd1 = D8(p + 172);
    return 2;
}
int decode_bdsephem(unsigned char* buff, GPSEPH* beph) {
    int prn;unsigned char* p = buff + 28;
    prn = UI4(p);
    if (prn<1 || prn>MAXBDSNUM)return0;
    }
    GPSEPH* eph = beph + prn - 1;
    eph->PRN = prn;
    eph->System = BDS;
    eph->TOC.Week = eph->TOE.Week = UI4(p + 4);
    eph->TOE.SecOfWeek = UI4(p + 72);
    eph->TOC.SecOfWeek = UI4(p + 40);
    eph->a0 = D8(p + 44);
    eph->a1 = D8(p + 52);
    eph->a2 = D8(p + 60);
    eph->IODE = UI4(p + 68);
    eph->IODC = UI4(p + 36);
    eph->sqrt_A = D8(p + 76);
    eph->M0 = D8(p + 108);
```

```
19.     eph->e = D8(p + 84);
20.     eph->OMEGA = D8(p + 116);
21.     eph->i0 = D8(p + 132);
22.     eph->omega = D8(p + 92);
23.     eph->Cic = D8(p + 180);
24.     eph->Cis = D8(p + 188);
25.     eph->Crc = D8(p + 164);
26.     eph->Crs = D8(p + 172);
27.     eph->Cuc = D8(p + 148);
28.     eph->Cus = D8(p + 156);
29.     eph->deltaN = D8(p + 100);
30.     eph->OMEGA_dot = D8(p + 124);
31.     eph->SVhealth = UI4(p + 16);
32.     eph->IDOT = D8(p + 140);
33.     eph->tgd1 = D8(p + 20);
34.     eph->tgd2 = D8(p + 28);
35.     return 3;
36. }
```

2.3 卫星位置计算

2.3.1 GPS 卫星

2.3.1.1 卫星位置与速度计算公式

（1） 计算轨道长半轴：

$$A = \left(\sqrt{A}\right)^2 \tag{27}$$

（2） 计算平均运动角速度：

$$n_0 = \sqrt{\frac{\mu}{A^3}} \tag{28}$$

（3） 计算相对于星历参考历元的时间：

$$t_k = t - t_{oe} \tag{29}$$

（4） t_k的取值判断：

$$\begin{cases} t_k = t_k - 604800，\ 当 t_k > 302400 \\ t_k = t_k + 604800, 当 t_k < -302400 \\ t_k = t_k, 其它情况 \end{cases} \tag{30}$$

（5） 对平均运动角速度进行改正：

$$n = n_0 + \Delta n \tag{31}$$

（6） 计算平近点角：

$$M_k = M_0 + nt_k \tag{32}$$

（7） 计算偏近点角：

$$M_k = E_k - e\sin E_k \tag{33}$$

（8） 计算真近点角：

$$v_k = \arctan\left(\frac{\sin v_k}{\cos v_k}\right) \tag{34}$$

$$v_k = \arctan\left(\frac{\sqrt{1-e^2}\sin E_k /(1-e\cos E_k)}{(\cos E_k - e)/(1-e\cos E_k)}\right) \tag{35}$$

（9） 计算升交角距：

$$\Phi_k = v_k + \omega \tag{36}$$

（10） 计算升交角距的改正数：

$$\delta u_k = C_{us}\sin 2\,\Phi_k + C_{uc}\cos 2\,\Phi_k \tag{37}$$

（11） 计算向径的改正数：

$$\delta r_k = C_{rs}\sin 2\,\Phi_k + C_{rc}\cos 2\,\Phi_k \tag{38}$$

（12） 计算轨道倾角改正数：

$$\delta i_k = C_{is}\sin 2\,\Phi_k + C_{ic}\cos 2\,\Phi_k \tag{39}$$

（13） 计算经过改正的升交角距：

$$u_k = \Phi_k + \delta u_k \tag{40}$$

（14） 计算经过改正的向径：

$$r_k = A(1 - e\cos E_k) + \delta r_k \tag{41}$$

（15） 计算经过改正的轨道倾角：

$$i_k = i_0 + \delta i_k + (IDOT)\cdot t_k \tag{42}$$

（16） 计算卫星在轨道平面上的x方向坐标：

$$x_k' = r_k\cos u_k \tag{43}$$

（17） 计算卫星在轨道平面上的y方向坐标：

$$y_k' = r_k \sin u_k \tag{44}$$

（18） 计算改正后的升交点经度：

$$\Omega_k = \Omega_0 + \left(\dot{\Omega} - \dot{\Omega}_e\right) \cdot t_k - \dot{\Omega}_e \cdot t_{oe} \tag{45}$$

（19） 计算卫星在地固坐标系下的x坐标：

$$x_k = x_k' \cos \Omega_k - y_k' \cos i_k \sin \Omega_k \tag{46}$$

（20） 计算卫星在地固坐标系下的y坐标：

$$y_k = x_k' \sin \Omega_k + y_k' \cos i_k \cos \Omega_k \tag{46}$$

（21） 计算卫星在地固坐标系下的z坐标：

$$z_k = y_k' \sin i_k \tag{47}$$

2.3.1.2 卫星钟差计算公式

（1） 卫星钟差改正：

$$\Delta t_{sv} = ClkBias + ClkDrift \cdot (t - t_{oc}) + ClkDriftRate \cdot (t - t_{oc})^2 + \Delta t_r \tag{48}$$

（2） 相对论效应改正：

$$\Delta t_r = Fe\sqrt{A} \sin E_k \tag{49}$$

（3） 常数F：

$$F = \frac{-2\sqrt{\mu}}{c^2} = -4.442807633 \cdot 10^{-10} \text{sec} \cdot \text{m}^{-\frac{1}{2}} \tag{50}$$

其中：t为观测时刻，Δt_r为相对论效应改正。

2.3.1.3 卫星钟速计算公式

（1） 计算偏近点角变化率：

$$\dot{E}_k = \frac{n}{1 - e \cos E_k} \tag{51}$$

（2） 计算升交角距变化率：

$$\dot{\Phi}_k = \frac{\sqrt{1-e^2}}{1-e\cos E_k}\dot{E}_k \tag{52}$$

（3） 计算经过改正的升交角距变化率：

$$\dot{u}_k = 2(C_{us}\cos 2\,\Phi_k - C_{uc}\sin 2\,\Phi_k)\dot{\Phi}_k + \dot{\Phi}_k \tag{53}$$

（4） 计算经过改正的向径变化率：

$$\dot{r}_k = Ae\sin E_k\,\dot{E}_k + 2(C_{rs}\cos 2\,\Phi_k - C_{rc}\sin 2\,\Phi_k)\dot{\Phi}_k \tag{54}$$

（5） 计算经过改正的轨道倾角变化率：

$$i_k = i + 2(C_{is}\cos 2\,\Phi_k - C_{ic}\sin 2\,\Phi_k)\dot{\Phi}_k \tag{55}$$

（6） 计算升交点经度变化率：

$$\dot{\Omega}_k = \dot{\Omega} - \dot{\Omega}_e \tag{56}$$

（7） 计算速度转换矩阵：

$$\dot{R} = \begin{bmatrix} \cos\Omega_k & -\sin\Omega_k\cos i_k & -({x_k}'\sin\Omega_k + {y_k}'\cos\Omega_k\cos i_k) & {y_k}'\sin\Omega_k\sin i_k \\ \sin\Omega_k & \cos\Omega_k\cos i_k & ({x_k}'\cos\Omega_k - {y_k}'\sin\Omega_k\cos i_k) & -{y_k}'\cos\Omega_k\sin i_k \\ 0 & \sin i_k & 0 & {y_k}'\cos i_k \end{bmatrix} \tag{57}$$

（8） 计算卫星在轨道平面上的x方向坐标：

$$x_k' = r_k\cos u_k \tag{58}$$

（9） 计算卫星在轨道平面上的y方向坐标：

$$y_k' = r_k\sin u_k \tag{59}$$

（10） 计算卫星在轨道平面上的x方向速度：

$$\dot{x}_k' = \dot{r}_k\cos u_k - r_k\dot{u}_k\sin u_k \tag{60}$$

（11） 计算卫星在轨道平面上的y方向速度：

$$\dot{y}_k' = \dot{r}_k\sin u_k + r_k\dot{u}_k\cos u_k \tag{61}$$

（12） 计算卫星在地固坐标系下的速度：

$$\begin{bmatrix}\dot{x}_k\\ \dot{y}_k\\ \dot{z}_k\end{bmatrix}=\dot{R}\begin{bmatrix}\dot{x}_k{}'\\ \dot{y}_k{}'\\ \dot{\Omega}_k\\ i_k\end{bmatrix} \tag{62}$$

（13） 卫星钟速改正：

$$\frac{d(\Delta t_{sv})}{dt}=ClkDrift+2ClkDriftRate\cdot(t-t_{oc})+\Delta\dot{t}_r \tag{63}$$

（14） 相对论效应改正导数：

$$\Delta\dot{t}_r=Fe\sqrt{A}\cos E_k\,\dot{E}_k \tag{64}$$

（15） 常数F：

$$F=\frac{-2\sqrt{\mu}}{c^2}=-4.442807633\cdot10^{-10}\ \mathrm{sec}\cdot\mathrm{m}^{-\frac{1}{2}} \tag{65}$$

其中，$\Delta\dot{t}_r$为相对论效应改正的导数。

2.3.1.4 程序实现

```
int CompGPSSatPVT(const int prn, const GPSTIME* t, const GPSEPH* geph, SATMIDRES* Mid) {
    const GPSEPH* eph = geph + prn - 1;
    GPSTIME GT = *t;
    GPSTIME GTOE = eph->TOE;
    double A = eph->sqrt_A * eph->sqrt_A;
    double n0 = sqrt(mu_GPS / (A * A * A));
    double tk = GetDiffTime(>, >OE);
    if (fabs(tk) > 7500.0 || eph->SVhealth != 0 || eph->PRN == 0) {
        Mid->Valid = false;
        return 0;
    }
    double n = n0 + eph->deltaN;
    double Mk = eph->M0 + n * tk;
    double Ek = Mk;
    double Ek_ref = Mk;
    for (int i = 0; i < 10; i++) {
        Ek_ref = Ek;
        Ek = Mk + eph->e * sin(Ek_ref);
        if (fabs(Ek - Ek_ref) < 1E-10) break;
    }
    double vk = atan2((sqrt(1 - eph->e * eph->e)*sin(Ek) ), ((cos(Ek) - eph->e)));
```

```
22.   double fi_k = vk + eph->omega;
23.   double delta_uk = eph->Cus * sin(2 * fi_k) + eph->Cuc * cos(2 * fi_k);
24.   double delta_rk = eph->Crs * sin(2 * fi_k) + eph->Crc * cos(2 * fi_k);
25.   double delta_ik = eph->Cis * sin(2 * fi_k) + eph->Cic * cos(2 * fi_k);
26.   double uk = fi_k + delta_uk;
27.   double rk = A * (1 - eph->e * cos(Ek)) + delta_rk;
28.   double ik = eph->i0 + delta_ik + eph->IDOT * tk;
29.   double xk1 = rk * cos(uk);
30.   double yk1 = rk * sin(uk);
31.   double OMEGA_k = eph->OMEGA + (eph->OMEGA_dot - OMEGA_e_dot_GPS) * tk - OMEGA_e_dot_GPS * eph->TOE.SecO
      fWeek;
32.   // 计算在地固坐标系下的位置
33.   Mid->SatPos[0] = xk1 * cos(OMEGA_k) - yk1 * cos(ik) * sin(OMEGA_k);
34.   Mid->SatPos[1] = xk1 * sin(OMEGA_k) + yk1 * cos(ik) * cos(OMEGA_k);
35.   Mid->SatPos[2] = yk1 * sin(ik);
36.   //GPS 钟差改正
37.   double F = -2 * sqrt(mu_GPS) / (CLIGHT*CLIGHT);
38.   double delta_t = F * eph->e * eph->sqrt_A * sin(Ek);
39.   Mid->SatClkOft += delta_t;
40.   //GPS 钟速计算
41.   double delta_tr_dot = F * eph->e * eph->sqrt_A * cos(Ek) * (n / (1 - eph->e * cos(Ek)));
42.   Mid->SatClkSft += delta_tr_dot;
43.   /****GPS 运动速度解算*********/
44.   double Ek_dot = n / (1 - eph->e * cos(Ek));
45.   double fi_k_dot = sqrt(1 - eph->e * eph->e) * Ek_dot / (1 - eph->e * cos(Ek));
46.   double uk_dot = 2 * (eph->Cus * cos(2 * fi_k) - eph->Cuc * sin(2 * fi_k)) * fi_k_dot + fi_k_dot;
47.   double rk_dot = eph->sqrt_A * eph->sqrt_A * eph->e * sin(Ek) * Ek_dot + 2 * (eph->Crs * cos(2 * fi_k) - eph->Crc * sin(2 * fi_k)
      ) * fi_k_dot;
48.   double ik_dot = eph->IDOT + 2 * (eph->Cis * cos(2 * fi_k) - eph->Cic * sin(2 * fi_k)) * fi_k_dot;
49.   double OMEGA_k_dot = eph->OMEGA_dot - OMEGA_e_dot_GPS;
50.   Matrix<double, 3, 4> R;
51.   R <<
52.       cos(OMEGA_k), -sin(OMEGA_k) * cos(ik), -
      (xk1 * sin(OMEGA_k) + yk1 * cos(OMEGA_k) * cos(ik)), yk1* sin(OMEGA_k)* sin(ik),
53.       sin(OMEGA_k), cos(OMEGA_k)* cos(ik), (xk1 * cos(OMEGA_k) - yk1 * sin(OMEGA_k) * cos(ik)), -
      yk1 * cos(OMEGA_k) * sin(ik),
54.       0, sin(ik), 0, yk1* cos(ik);
55.    double xk1_dot = rk_dot * cos(uk) - rk * uk_dot * sin(uk);
56.   double yk1_dot = rk_dot * sin(uk) + rk * uk_dot * cos(uk);
57.   double xk_dot, yk_dot, zk_dot;
58.   Vector3d V/*(xk_dot, yk_dot, zk_dot)*/;
59.   Vector4d DOT(xk1_dot, yk1_dot, OMEGA_k_dot, ik_dot);
```

```
60.     V = R * DOT;
61.     Mid->SatVel[0] = V(0);
62.     Mid->SatVel[1] = V(1);
63.     Mid->SatVel[2] = V(2);
64.     Mid->Valid = true;
65.     Mid->Tgd1 = eph->tgd1 * CLIGHT;
66.     return 1;
67. }
```

2.3.2 BDS 卫星

2.3.2.1 卫星位置与速度计算公式

（1） 计算真近点角的正弦值：

$$\sin v_k = \frac{\sqrt{1-e^2}\sin E_k}{1-e\cos E_k} \tag{66}$$

（2） 计算真近点角的余弦值：

$$\cos v_k = \frac{\cos E_k - e}{1-e\cos E_k} \tag{67}$$

（3） 计算纬度幅角：

$$\phi_k = v_k + \omega \tag{68}$$

（4） 计算纬度幅角改正项：

$$\delta u_k = C_{us}\sin(2\phi_k) + C_{uc}\cos(2\phi_k) \tag{69}$$

（5） 计算径向改正项：

$$\delta r_k = C_{rs}\sin(2\phi_k) + C_{rc}\cos(2\phi_k) \tag{70}$$

（6） 计算轨道倾角改正项：

$$\delta i_k = C_{is}\sin(2\phi_k) + C_{ic}\cos(2\phi_k) \tag{71}$$

（7） 计算改正后的纬度幅角：

$$u_k = \phi_k + \delta u_k \tag{72}$$

（8） 计算改正后的径向距离：

$$r_k = A(1 - e\cos E_k) + \delta r_k \tag{73}$$

（9） 计算改正后的轨道倾角：

$$i_k = i_0 + IDOT \cdot t_k + \delta i_k \tag{74}$$

（10） 计算卫星在轨道平面内的x坐标：

$$x_k = r_k \cos u_k \tag{75}$$

（11） 计算卫星在轨道平面内的y坐标：

$$y_k = r_k \sin u_k \tag{76}$$

（12） 计算 MEO 和 IGSO 卫星的历元升交点经度：

$$\Omega_k = \Omega_0 + (\dot{\Omega} - \dot{\Omega}_e)t_k - \dot{\Omega}_e t_{oe} \tag{77}$$

（13） 计算 MEO 和 IGSO 卫星在 BDCS 坐标系中的X坐标：

$$X_k = x_k \cos\Omega_k - y_k \cos i_k \sin\Omega_k \tag{78}$$

（14） 计算 MEO 和 IGSO 卫星在 BDCS 坐标系中的Y坐标：

$$Y_k = x_k \sin\Omega_k + y_k \cos i_k \cos\Omega_k \tag{79}$$

（15） 计算 MEO 和 IGSO 卫星在 BDCS 坐标系中的Z坐标：

$$Z_k = y_k \sin i_k \tag{80}$$

（16） 计算 GEO 卫星的历元升交点经度：

$$\Omega_k = \Omega_0 + \dot{\Omega} t_k - \dot{\Omega}_e t_{oe} \tag{81}$$

（17） 计算 GEO 卫星在自定义坐标系中的X坐标：

$$X_{GK} = x_k \cos\Omega_k - y_k \cos i_k \sin\Omega_k \tag{82}$$

（18） 计算 GEO 卫星在自定义坐标系中的Y坐标：

$$Y_{GK} = x_k \sin\Omega_k + y_k \cos i_k \cos\Omega_k \tag{83}$$

（19） 计算 GEO 卫星在自定义坐标系中的Z坐标：

$$Z_{GK} = y_k \sin i_k \tag{84}$$

（20） 计算 GEO 卫星在 BDCS 坐标系中的坐标：

$$\begin{bmatrix} X_k \\ Y_k \\ Z_k \end{bmatrix} = R_z(\dot{\Omega}_e t_k) R_x(-5°) \begin{bmatrix} X_{GK} \\ Y_{GK} \\ Z_{GK} \end{bmatrix} \tag{85}$$

（21） 绕X轴旋转矩阵：

$$R_x(\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{bmatrix} \tag{86}$$

（22） 绕Z轴旋转矩阵：

$$R_z(\varphi) = \begin{bmatrix} \cos\varphi & \sin\varphi & 0 \\ -\sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{87}$$

（23） 计算 MEO 和 IGSO 卫星的升交点经度变化率：

$$\dot{\Omega}_k = \dot{\Omega} - \dot{\Omega}_e \tag{88}$$

（24） 计算 MEO 和 IGSO 卫星在 BDCS 坐标系中的速度：

$$\begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix} = \dot{R} \begin{bmatrix} \dot{x}_k{}' \\ \dot{y}_k{}' \\ \dot{\Omega}_k \\ i_k \end{bmatrix} \tag{89}$$

（25） 计算 GEO 卫星的升交点经度变化率：

$$\dot{\Omega}_k = \dot{\Omega} \tag{90}$$

（26） 计算 GEO 卫星在 BDCS 坐标系中的位置：

$$\begin{bmatrix} X_k \\ Y_k \\ Z_k \end{bmatrix} = R_z(\dot{\Omega}_e t_k) R_x(-5°) \begin{bmatrix} X_{GK} \\ Y_{GK} \\ Z_{GK} \end{bmatrix} \tag{91}$$

（27） 计算 GEO 卫星在 BDCS 坐标系中的速度：

$$\frac{d}{dt}\begin{bmatrix}X_k\\Y_k\\Z_k\end{bmatrix}=R_z(\dot{\Omega}_e t_k)R_x(-5°)\frac{d}{dt}\begin{bmatrix}X_{GK}\\Y_{GK}\\Z_{GK}\end{bmatrix}+\dot{R}_z(\dot{\Omega}_e t_k)R_x(-5°)\begin{bmatrix}X_{GK}\\Y_{GK}\\Z_{GK}\end{bmatrix} \tag{92}$$

2.3.2.2 卫星钟差与钟速

计算方法与 GPS 卫星相同，需要注意的是地球自转角速度需要使用 CGCS2000 坐标系下的值。

2.3.2.3 卫星设备延迟改正

（1） 对使用 B1I 信号的单频用户进行修正：

$$(\Delta t_{sv})_{B1I}=\Delta t_{sv}-T_{GD1} \tag{93}$$

（2） 对使用 B2I 信号的单频用户进行修正：

$$(\Delta t_{sv})_{B2I}=\Delta t_{sv}-T_{GD2} \tag{94}$$

Δt_{sv}为卫星测距码相位时间偏移

（3） 对 B1I 和 B3I 双频用户进行电离层组合伪距修正：

$$PR=\frac{PR_{B3I}-k_{1,3}(f)\cdot PR_{B1I}}{1-k_{1,3}(f)}+\frac{C\cdot k_{1,3}(f)\cdot T_{GD1}}{1-k_{1,3}(f)} \tag{95}$$

其中，$k_{1,3}(f)=\frac{f_1^2}{f_3^2}$，$PR$为经过电离层修正后的伪距，$PR_{B1I}$为 B1I 信号的观测伪距，$PR_{B3I}$为 B3I 信号的观测伪距，$T_{GD1}$为 B1I 信号的星上设备时延差，$C=2.99792458\times10^8\mathrm{m/s}$。

2.3.2.4 程序实现

```
int CompBDSSatPVT(const int prn, const GPSTIME* t, const GPSEPH* beph, SATMIDRES* Mid) {
    const GPSEPH* eph = beph + prn - 1;
    GPSTIME BT = *t;
    BT.Week -= 1356;
    BT.SecOfWeek -= 14;
    if (BT.SecOfWeek < 0.0) {
        BT.SecOfWeek += 604800.0;
        BT.Week -= 1;
    }
    GPSTIME BTOE = eph->TOE;
    double A = eph->sqrt_A * eph->sqrt_A;
    double n0 = sqrt(mu_BDS / (A * A * A));
    double tk = GetDiffTime(&BT, &BTOE);
    if (fabs(tk) > 3900.0 || eph->SVhealth != 0 || eph->PRN == 0) {
        Mid->Valid = false;
        return 0;
    }
    double n = n0 + eph->deltaN;
    double Mk = eph->M0 + n * tk;
    double Ek = Mk;
    double Ek_ref = Mk;
    for (int i = 0; i < 10; i++) {
        Ek_ref = Ek;
        Ek = Mk + eph->e * sin(Ek_ref);
```

```
25.         if (fabs(Ek - Ek_ref) < 1E-10) break;
26.     }
27.     double vk = atan2((sqrt(1 - eph->e * eph->e)*sin(Ek)), ((cos(
    Ek) - eph->e)));
28.     double fi_k = vk + eph->omega;
29.     double delta_uk = eph->Cus * sin(2 * fi_k) + eph->Cuc * cos
    (2 * fi_k);
30.     double delta_rk = eph->Crs * sin(2 * fi_k) + eph->Crc * cos(
    2 * fi_k);
31.     double delta_ik = eph->Cis * sin(2 * fi_k) + eph->Cic * cos(2
     * fi_k);
32.     double uk = fi_k + delta_uk;
33.     double rk = A * (1 - eph->e * cos(Ek)) + delta_rk;
34.     double ik = eph->i0 + delta_ik + eph->IDOT * tk;
35.     double xk = rk * cos(uk);
36.     double yk = rk * sin(uk);
37.     double Ek_dot = n / (1 - eph->e * cos(Ek));
38.     double fi_k_dot = sqrt(1 - eph->e * eph->e) * Ek_dot / (1 - ep
    h->e * cos(Ek));
39.     double uk_dot = 2 * (eph->Cus * cos(2 * fi_k) - eph->Cuc * s
    in(2 * fi_k)) * fi_k_dot + fi_k_dot;
40.     double rk_dot = eph->sqrt_A * eph->sqrt_A * eph->e * sin(E
    k) * Ek_dot + 2 * (eph->Crs * cos(2 * fi_k) - eph->Crc * sin(2 *
    fi_k)) * fi_k_dot;
41.     double ik_dot = eph->IDOT + 2 * (eph->Cis * cos(2 * fi_k) -
    eph->Cic * sin(2 * fi_k)) * fi_k_dot;
42.     double OMEGA_k_dot = eph->OMEGA_dot - OMEGA_e_d
    ot_BDS;
43.     double xk1_dot = rk_dot * cos(uk) - rk * uk_dot * sin(uk);
44.     double yk1_dot = rk_dot * sin(uk) + rk * uk_dot * cos(uk);
45.     // 升交点赤经
46.     if ((prn >= 1 && prn <= 5) || (prn >= 59 && prn <= 63)) {
47.         double OMEGA_k = eph->OMEGA + eph->OMEGA_dot
    * tk - OMEGA_e_dot_BDS * eph->TOE.SecOfWeek;
48.         double xgk = xk * cos(OMEGA_k) - yk * cos(ik) * sin(O
    MEGA_k);
49.         double ygk = xk * sin(OMEGA_k) + yk * cos(ik) * cos(O
    MEGA_k);
50.         double zgk = yk * sin(ik);
51.         Vector3d POS_GK(xgk, ygk, zgk);
52.         Vector3d POS;
53.         POS = Rz(OMEGA_e_dot_BDS * tk) * Rx(Degree2Rad(-
    5.0)) * POS_GK;
54.         Mid->SatPos[0] = POS(0);
55.         Mid->SatPos[1] = POS(1);
56.         Mid->SatPos[2] = POS(2);
57.         /**********GEO 速度解算************/
58.         Matrix<double, 3, 4> R;
59.         R <<
60.             cos(OMEGA_k), -sin(OMEGA_k) * cos(ik), -
    (xk * sin(OMEGA_k) + yk * cos(OMEGA_k) * cos(ik)), yk* sin
    (OMEGA_k)* sin(ik),
61.             sin(OMEGA_k), cos(OMEGA_k)* cos(ik), (xk * cos(O
    MEGA_k) - yk * sin(OMEGA_k) * cos(ik)), -
    yk * cos(OMEGA_k) * sin(ik),
62.             0, sin(ik), 0, yk* cos(ik);
63.         Vector3d V_GK;
64.         double OMEGA_k_dot_geo = eph->OMEGA_dot;
65.         Vector4d DOT(xk1_dot, yk1_dot, OMEGA_k_dot_geo, ik_
    dot);
66.         V_GK = R * DOT;
67.         Vector3d V;
68.         V = Rz(OMEGA_e_dot_BDS * tk) * Rx(Degree2Rad(-
    5.0)) * V_GK + OMEGA_e_dot_BDS * Rz_dot(OMEGA_e_do
    t_BDS * tk) * Rx(Degree2Rad(-5.0)) * POS_GK;
69.         Mid->SatVel[0] = V(0);
70.         Mid->SatVel[1] = V(1);
71.         Mid->SatVel[2] = V(2);
72.     }
73.     else {
74.         // 计算改正后的升交点赤经
75.         double OMEGA_k = eph->OMEGA + (eph->OMEGA_dot
     - OMEGA_e_dot_BDS) * tk - OMEGA_e_dot_BDS * eph->T
    OE.SecOfWeek;
76.         // 计算在地固坐标系下的位置
77.         Mid->SatPos[0] = xk * cos(OMEGA_k) - yk * cos(ik) * sin
    (OMEGA_k);
78.         Mid->SatPos[1] = xk * sin(OMEGA_k) + yk * cos(ik) * co
    s(OMEGA_k);
79.         Mid->SatPos[2] = yk * sin(ik);
80.         Matrix<double, 3, 4> R;
81.         R <<
82.             cos(OMEGA_k), -sin(OMEGA_k) * cos(ik), -
    (xk * sin(OMEGA_k) + yk * cos(OMEGA_k) * cos(ik)), yk* sin
    (OMEGA_k)* sin(ik),
83.             sin(OMEGA_k), cos(OMEGA_k)* cos(ik), (xk * cos(O
    MEGA_k) - yk * sin(OMEGA_k) * cos(ik)), -
    yk * cos(OMEGA_k) * sin(ik),
84.             0, sin(ik), 0, yk* cos(ik);
85.         double xk_dot, yk_dot, zk_dot;
```

```
        Vector3d V/*(xk_dot, yk_dot, zk_dot)*/;
        Vector4d DOT(xk1_dot, yk1_dot, OMEGA_k_dot, ik_dot);

        V = R * DOT;
        Mid->SatVel[0] = V(0);
        Mid->SatVel[1] = V(1);
        Mid->SatVel[2] = V(2);
    }
    //BDS 钟差改正
    double F = -2 * sqrt(mu_BDS) / (CLIGHT * CLIGHT);
    double delta_t = F * eph->e * eph->sqrt_A * sin(Ek);
    double k1_3 = FG1_BDS * FG1_BDS / (FG3_BDS * FG3_BDS);
    double delta_tsv = k1_3 * eph->tgd1 / (1 - k1_3);
    Mid->SatClkOft = Mid->SatClkOft + delta_t + delta_tsv;
    //BDS 钟速计算
    double delta_tr_dot = F * eph->e * eph->sqrt_A * cos(Ek) * (n / (1 - eph->e * cos(Ek)));
    Mid->SatClkSft += delta_tr_dot;
    Mid->Valid = true;
    Mid->Tgd1 = eph->tgd1 * CLIGHT;
    Mid->Tgd2 = eph->tgd2 * CLIGHT;
    return 1;
}
```

2.3.3 星历检查

2.3.3.1 计算公式

首先计算时间差：

$$\Delta t = \left(t_{obs}^{week} - t_{TOC}^{week}\right) \cdot 604800 + \left(t_{obs}^{sec} - t_{TOC}^{sec}\right) \tag{96}$$

对于 BDS 系统，需要进行时间系统转换：

$$t_{BDT} = t_{GPST} - 1356\ 周 - 14\ 秒 \tag{97}$$

进行有效性判断：

$$\begin{cases} Valid = True, & |\Delta t| \le T_{max} 且 SVHealth = 0 \\ Valid = False, & Others \end{cases} \tag{98}$$

GPS 系统星历有效期T_{max}为 2 小时，这里我设置 7500s，BDS 系统星历有效期T_{max}为 1 小时，约等于 3900s。

2.3.3.2 程序实现

```
bool CompSatClkOff(const int prn, const GNSSSYS sys, const GPSTIME* t, GPSEPH* geph, GPSEPH* beph, SATMIDRES* Mid) {
    double dt, Limt=7500;
    GPSTIME GT;
    GPSEPH* eph= geph + prn - 1;
    if (sys == GPS) {
        eph = geph + prn - 1;
        Limt = 7500.0;
        GT = *t;
    }
```

```
    else if (sys == BDS) {
        eph = beph + prn - 1;
        Limt = 3900.0;
        GT.Week = t->Week - 1356;
        GT.SecOfWeek = t->SecOfWeek - 14.0;
        if (GT.SecOfWeek < 0.0) {
            GT.SecOfWeek += 604800.0;
            GT.Week -= 1;
        }
    }
    dt = GetDiffTime(>, &eph->TOC);
    if (fabs(dt) > Limt || eph->SVhealth != 0) return false;
    Mid->SatClkOft = eph->a0 + eph->a1 * dt + eph->a2 * dt * dt;
    Mid->SatClkSft = eph->a1 + 2 * eph->a2 * dt;
    Mid->Valid = true;
    return true;
}
```

2.4 误差改正

2.4.1 地球自转改正

2.4.1.1 计算公式

$$X = R_Z(\omega_e \Delta t)\begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix} \tag{99}$$

$$\dot{X} = R_Z(\omega_e \Delta t)\begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix} \tag{100}$$

$$\Delta t = \frac{\rho}{c} \tag{101}$$

$$\rho = \sqrt{(x^s - x_r)^2 + (y^s - y_r)^2 + (z^s - z_r)^2} \tag{102}$$

$$R_Z(\alpha) = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{103}$$

其中，ω_e为地球自转角速度，Δt为信号传播时间，不能直接用伪距代替，ρ为卫星与接收机之间的几何距离，c为光速

2.4.1.2 程序实现

```
1.  void EarthRotationCorrection(SATMIDRES* Mid, double UserPos[3], double Omegae) {
2.      double rou, dt;
3.      rou = sqrt(SQR(Mid->SatPos[0] - UserPos[0]) + SQR(Mid->SatPos[1] - UserPos[1]) + SQR(Mi
    d->SatPos[2] - UserPos[2]));
4.      dt = rou / CLIGHT;
5.      Matrix<double, 3, 1> SatPOS;
6.      SatPOS <<
7.          Mid->SatPos[0],
8.          Mid->SatPos[1],
9.          Mid->SatPos[2];
10.     Matrix<double, 3, 1> SatVEL;
11.     SatVEL <<
12.         Mid->SatVel[0],
13.         Mid->SatVel[1],
14.         Mid->SatVel[2];
15.     SatPOS = Rz(Omegae * dt) * SatPOS;
16.     SatVEL = Rz(Omegae * dt) * SatVEL;
17.     Mid->SatPos[0] = SatPOS[0];
18.     Mid->SatPos[1] = SatPOS[1];
19.     Mid->SatPos[2] = SatPOS[2];
20.
21.     Mid->SatVel[0] = SatVEL[0];
22.     Mid->SatVel[1] = SatVEL[1];
23.     Mid->SatVel[2] = SatVEL[2];
24. }
```

2.4.2 大气误差

2.4.2.1 电离层延迟

本软件使用的是双频无电离层组合（Ionosphere-Free, IF）改正，已消除电离层延迟。

2.4.2.2 对流层延迟

（1） 计算公式

$$\Delta_{trop} = \Delta_d + \Delta_w = \frac{K_d \cdot p}{\sin\left(\sqrt{E^2 + 6.25}\right) + 2.25} + \frac{K_w \cdot e}{\sin\left(\sqrt{E^2 + 2.25}\right)} \tag{104}$$

$$K_d = 155.2 \times 10^{-7} \cdot \frac{p}{T}(h_d - H) \tag{105}$$

$$K_w = 155.2 \times 10^{-7} \cdot \frac{4810}{T^2} e(h_w - H) \tag{106}$$

$$h_d = 40136 + 148.72(T_0 - 273.16) \tag{107}$$

$$h_w = 11000 \tag{108}$$

$$e = RH \cdot exp(-37.2465 + 0.213166T - 0.000256908T^2) \tag{109}$$

$$T = T_0 - 0.0065 \cdot (H - H_0) \tag{110}$$

$$p = p_0 \cdot \left(1 - 0.0000226 \cdot (H - H_0)\right)^{5.225} \tag{111}$$

$$RH = RH_0 \cdot exp\left(-0.0006396(H - H_0)\right) \tag{112}$$

其中，H_0为参考面的高度，T_0、p_0和RH_0分别为参考面上的干温、气压和相对湿度，H为测站高度，T、p和RH分别为测站上的干温、气压和相对湿度，E为卫星相对于测站的高度角。

（2） 程序实现

```
1.  double HopefieldTropCorr(double H, double E) {
2.      double RH, p, T, e, hw, hd, Kw, Kd, dTrop;
3.      if (H < 0) return 0;
4.      if (H < -1000.0 || H > 50000.0) return 0.0;
5.      if (E <= 0.0) return 0.0;
6.      RH = RH0 * exp(-0.0006396 * (H - H0));
7.      p = P0 * pow((1 - 0.0000226 * (H - H0)), 5.225);
8.      T = T0 - 0.0065 * (H - H0);
9.      e = RH * exp(-37.2465 + 0.213166 * T - 0.000256908 * T * T);
10.     hw = 11000;
11.     hd = 40136 + 148.72 * (T0 - 273.16);
12.     Kw = 155.2e-7 * 4810 * e * (hw - H) / SQR(T);
13.     Kd = 155.2e-7 * p * (hd - H) / T;
14.     double Edeg = E * Rad2Deg;
15.     double md = 1.0 / sin(Deg2Rad * sqrt(SQR(Edeg) + 6.25));
```

```
    double mw = 1.0 / sin(Deg2Rad * sqrt(SQR(Edeg) + 2.25));
    return Kd * md + Kw * mw;
}
```

2.4.3 粗差探测

（1） 计算公式

计算 MW 组合：

$$L_{MW} = \frac{1}{f_1 - f_2}(f_1 L_1 - f_2 L_2) - \frac{1}{f_1 + f_2}(f_1 P_1 + f_2 P_2) \tag{113}$$

计算 GF 组合：

$$L_{GF} = L_1 - L_2 = \lambda_1 \varphi_1 - \lambda_2 \varphi_2 \tag{114}$$

判断 GF 组合是否满足条件：

$$|L_{GF}(t_k) - L_{GF}(t_{k-1})| < \delta_{GF} = 0.05m \tag{115}$$

判断 MW 组合是否满足条件：

$$\left|L_{MW}(t_k) - \overline{L}_{MW}(t_{k-1})\right| < \delta_{MW} = 3.00m \tag{116}$$

若满足条件，即没有伪距粗差和相位周跳，计算 MW 组合的平滑值，将观测值设置为可用；否则，将观测值设置为不可用，重置L_{GF}和L_{MW}平滑值：

$$\overline{L}_{MW}(t_k) = \frac{(k-1) \times \overline{L}_{MW}(t_{k-1}) + L_{MW}(t_k)}{k} \tag{117}$$

（2） 程序实现

```
void DetectOutlier(EPOCHOBSDATA* obs) {
    MWGF CurComObs[MAXCHANNUM];
    double Lmw, Lgf, Pif, f1, f2 , dMW, dGF;
    memset(CurComObs, 0, sizeof(CurComObs));
    for (int i = 0; i < obs->SatNum; i++) {
        obs->SatObs[i].valid = false;
        if (fabs(obs->SatObs[i].P[0]) < 1e-5 || fabs(obs->SatObs[i].P[1]) < 1e-5
    || fabs(obs->SatObs[i].L[0]) < 1e-5 || fabs(obs->SatObs[i].L[1]) < 1e-5) continue;
        // 2.计算当前历元该卫星的 GF 和 MW 组合值
        if (obs->SatObs[i].System == GPS) {
            f1 = FG1_GPS;
            f2 = FG2_GPS;
        }
        else if (obs->SatObs[i].System == BDS) {
            f1 = FG1_BDS;
```

```
            f2 = FG3_BDS;
        }
        else continue;
        Lmw = (f1 * obs->SatObs[i].L[0] - f2 * obs->SatObs[i].L[1]) / (f1 - f2) -
            (f1 * obs->SatObs[i].P[0] + f2 * obs->SatObs[i].P[1]) / (f1 + f2);
        Lgf = obs->SatObs[i].L[0] - obs->SatObs[i].L[1];
        //对于可用的观测数据，计算伪距的 IF 组合观测值，用于 SPP
        Pif = (SQR(f1) * obs->SatObs[i].P[0] - SQR(f2) * obs->SatObs[i].P[1]) / (SQR(f1) - SQR(f2));
        // 记录当前历元的 MWGF
        CurComObs[i].GF = Lgf;
        CurComObs[i].MW = Lmw;
        CurComObs[i].PIF = Pif;
        CurComObs[i].Prn = obs->SatObs[i].Prn;
        CurComObs[i].System = obs->SatObs[i].System;
        CurComObs[i].n = 1;
        bool found = false;
        for (int j = 0; j < MAXCHANNUM; j++) {
            if (obs->SatObs[i].System == obs->CombObs[j].System &&
                obs->SatObs[i].Prn == obs->CombObs[j].Prn) {
                found = true;
                dGF = fabs(CurComObs[i].GF - obs->CombObs[j].GF);
                dMW = fabs(CurComObs[i].MW - obs->CombObs[j].MW);
                if (dGF < GF_Threshold && dMW < MW_Threshold) {
                    obs->SatObs[i].valid = true;
                    CurComObs[i].MW = ((CurComObs[i].n - 1) * obs->CombObs[j].MW + CurComObs[i].MW) / CurComObs[i].n;
                    CurComObs[i].n += 1;
                }
                else {
                    obs->SatObs[i].valid = false;
                    CurComObs[i].n = 0;
                    continue;
                    break;
                }
            }
            if (!found) {
                obs->SatObs[i].valid = true;
            }
        }
        memcpy(obs->CombObs, CurComObs, sizeof(CurComObs));
    }
}
```

2.5 单点定位与测速

2.5.1 信号发射时刻的卫星状态

（1） 计算公式

首先迭代计算某颗卫星j的信号发射时刻：

$$t_j^{trans} = t_{rec} - \frac{P_j}{c} - \delta t_j \tag{118}$$

$$\delta t_j = a_0 + a_1 \cdot \left(t_j^{trans} - t_{toc}\right) + a_2 \cdot \left(t_j^{trans} - t_{toc}\right)^2 \tag{119}$$

其中t_{rec}是接收机接收到信号的时刻；P_j为观测到的伪距；δt_j是卫星钟差；t_{toc}是广播星历的参考时刻；

a_0，a_1，a_2分别是钟差、钟漂和钟偏；迭代终止条件为：

$$\left|\delta t_j^{(k)} - \delta t_j^{(k-1)}\right| < 10^{-12} \tag{120}$$

然后计算卫星位置与速度：

$$\theta = w_e \cdot \frac{\rho_j - \rho_{user}}{c} \tag{121}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{122}$$

$$\begin{bmatrix} X^j \\ Y^j \\ Z^j \end{bmatrix}_{ECEF} = R_z(\theta) \cdot \begin{bmatrix} X^j \\ Y^j \\ Z^j \end{bmatrix}_{inertial} \tag{123}$$

$$\begin{bmatrix} V_X^j \\ V_Y^j \\ V_Z^j \end{bmatrix}_{ECEF} = R_z(\theta) \cdot \begin{bmatrix} V_X^j \\ V_Y^j \\ V_Z^j \end{bmatrix}_{inertial} \tag{124}$$

其中ρ是卫星与接收机之间的几何距离；w_e是地球自转角速度；r_{user}是用户接收机位置；$inertial$是原先计算的卫星的位置、速度。

（2） 程序实现

```
1.  void SatPVTCalculation(GPSEPH* geph, GPSEPH* beph, EPOCHOBSDATA* obs, SPPPOS* UserPos/*double UserPos[3]*/) {
2.      GPSTIME tr = obs->Time;   // 接收机钟的表面时，接收机接收到卫星信号的时刻
3.      GPSTIME ts;               // 卫星信号发射时刻
4.      double blh[3];
```

```
5.    double R, F;
6.    for (int i = 0; i < obs->SatNum; i++) {
7.        SATMIDRES* Mid = obs->SatPVT + i;     //Mid 指向当前卫星的 PVT 计算结果结构体
8.        ts.Week = tr.Week;
9.        ts.SecOfWeek = tr.SecOfWeek - obs->SatObs[i].P[0] / CLIGHT; //第一次没有卫星钟差
10.       if (!CompSatClkOff(obs->SatObs[i].Prn,obs->SatObs[i].System,&ts,geph,beph,Mid)) {
11.           Mid->Valid = false;
12.           continue;
13.       }
14.       ts.SecOfWeek = tr.SecOfWeek - obs->SatObs[i].P[0] / CLIGHT - obs->SatPVT[i].SatClkOft;
15.       if (!CompSatClkOff(obs->SatObs[i].Prn, obs->SatObs[i].System, &ts, geph, beph, Mid)) {
16.           Mid->Valid = false;
17.           continue;
18.       }
19.       ts.SecOfWeek = tr.SecOfWeek - obs->SatObs[i].P[0] / CLIGHT - obs->SatPVT[i].SatClkOft;
20.       if (obs->SatObs[i].System == GPS) {
21.           if (!CompGPSSatPVT(obs->SatObs[i].Prn,&ts,geph,Mid)) {
22.               Mid->Valid = false;
23.               continue;
24.           }
25.           EarthRotationCorrection(Mid, UserPos->Pos, Omega_WGS);
26.       }
27.       else if (obs->SatObs[i].System == BDS) {
28.           if (!CompBDSSatPVT(obs->SatObs[i].Prn,&ts,beph,Mid)) {
29.               Mid->Valid = false;
30.               continue;
31.           }
32.           EarthRotationCorrection(Mid, UserPos->Pos, Omega_BDS);
33.       }
34.       else {
35.           Mid->Valid = false;
36.           continue;
37.       }
38.       Mid->Valid = true;
39.       // 计算卫星的高度角和对流层改正
40.       if (obs->SatObs[i].System == GPS) {
41.           R = R_WGS84;
42.           F = F_WGS84;
43.       }
44.       else if (obs->SatObs[i].System == BDS) {
45.           R = R_CGS2K;
46.           F = F_CGS2K;
```

```
47.     }
48.     else continue;
49.     CompSatElAz(UserPos->Pos, obs->SatPVT[i].SatPos, R, F, &obs->SatPVT[i].Elevation, &obs->SatPVT[i].Azimuth);
50.     XYZToBLH(UserPos->Pos, blh, R, F);
51.     obs->SatPVT[i].TropCorr = HopefieldTropCorr(blh[2], obs->SatPVT[i].Elevation);
52.   }
53. }
```

2.5.2 单点定位

2.5.2.1 计算公式

对于第j颗卫星，观测模型为：

$$P_j^{PIF} = \rho_j + c \cdot \delta t_{sys(j)} - c \cdot \delta t_j + T_j + \delta_{ion}^{PIF} + \varepsilon_j \tag{125}$$

其中P_j^{PIF}是 PIF 组合伪距；ρ_j是卫地几何距离；$\delta t_{sys(j)}$是接收机钟差；δt_j是卫星钟差；T_j是对流层改正量；δ_{ion}^{PIF}是群延迟改正项，BDS 专用$\delta_{ion}^{PIF} = \frac{f_1^2}{f_1^2 - f_3^2} \cdot c \cdot T_{gd1}$；$\varepsilon_j$是测量噪声项。

接着对方程进行线性化：

$$x = [\Delta x, \Delta y, \Delta z, \Delta(c \cdot \delta t_{GPS}), \Delta(c \cdot \delta t_{BDS})]^T \tag{126}$$

$$w_j = P_j^{PIF} - \rho_j - \delta_{corr}^{PIF} \approx B_j \cdot x + v_j \tag{127}$$

$$B_j = \left[-\frac{X_j - X_R}{\rho_j} \quad -\frac{Y_j - Y_R}{\rho_j} \quad -\frac{Z_j - Z_R}{\rho_j} \quad \delta_{j \in GPS} \quad \delta_{j \in BDS} \right] \tag{128}$$

其中x是状态变量；w_j观测残差；B_j是伪距观测方程线性化后的观测矩阵的第j行系数向量。

接着进行最小二乘解运算：

$$x = \underset{n \times p}{\left(\underset{n \times p}{B^T} \underset{n \times n}{P} \underset{n \times p}{B} \right)^{-1}} \cdot \underset{n \times p}{B^T} \underset{n \times n}{P} \cdot \underset{n}{w} \tag{129}$$

其中若仅有 GPS 卫星历元时，裁剪B的第 5 列，此时列数 4；若仅有 BDS 卫星历元时，裁剪B的第 4 列，此时列数为 4；若同时有 GPS 与 BDS 卫星历元，不裁剪B的列数，此时列数为 5。

接着进行定位的迭代更新：

$$X^{k+1} = X^k + x_{1-3} \tag{130}$$

$$\delta t_{GPS}^{k+1} = \delta t_{GPS}^{k} + x_4 \tag{131}$$

$$\delta t_{BDS}^{k+1} = \delta t_{BDS}^{k} + x_5 \tag{132}$$

其中X^k为坐标；收敛判据为：$X^{k+1} - X^k < 10^{-7}m$；最大迭代次数为 15 次。

接着对定位精度进行评估：

$$\sigma_0 = \sqrt{\frac{(Bx - w)^T P (Bx - w)}{n - p}} \tag{133}$$

$$PDOP = \sqrt{q_{xx} + q_{yy} + q_{zz}} \tag{134}$$

其中σ_0是位置精度；PDOP 是位置精度因子。

2.5.2.2 程序实现

```
bool SPP(GPSEPH* geph, GPSEPH* beph, SPPPOS *UserPos, EPOCHOBSDATA* obs) {
    int i = 0;
    double blh[3], dPos[3],Rou , sigma2;
    int GPSNum = 0, BDSNum = 0, n;
    MatrixXd B(MAXCHANNUM, 5), BTB(5, 5), BTW(5, 1);
    VectorXd w(MAXCHANNUM), X(5),X0(5),V(MAXCHANNUM);
    if (fabs(UserPos->Pos[0]) < 1e-6 && fabs(UserPos->Pos[1]) < 1e-6 && fabs(UserPos->Pos[2]) < 1e-6) {
        //X0 << 1000.0, 1000.0, 1000.0, 0.0, 0.0;
        X0 << -2267335.1440, 5008648.2444, 3222378.1781, 0.0, 0.0;
    }
    else {
        X0 << UserPos->Pos[0], UserPos->Pos[1], UserPos->Pos[2],
            UserPos->RcvClk[0] * CLIGHT, UserPos->RcvClk[1] * CLIGHT;
    }
    //计算发射时刻的卫星位置和卫星钟差（包含地球自转改正）
    for(i = 0 ; i < 10 ; i++ ){     // 最多循环迭代 15 次
        X.setZero();
        B.setZero();
        BTB.setZero();
        BTW.setZero();
        w.setZero();
        V.setZero();
        UserPos->Pos[0] = X0(0);
        UserPos->Pos[1] = X0(1);
```

```
25.     UserPos->Pos[2] = X0(2);
26.     UserPos->RcvClk[0] = X0(3);
27.     UserPos->RcvClk[1] = X0(4);
28.     SatPVTCalculation(geph, beph, obs, UserPos);
29.     n = GPSNum = BDSNum = 0;
30.     sigma2 = 0;
31.     // 卫星位置计算失败、观测数据不完整或有粗差，不参与定位计算
32.     for (int j = 0; j < obs->SatNum; j++) {
33.         if (obs->SatPVT[j].Valid == false || obs->SatObs[j].valid == false) continue;
34.         // 保留初始位置，作为后面的计算的参考值
35.         for (int k = 0; k < 3; k++) {
36.             dPos[k] = X0(k) - obs->SatPVT[j].SatPos[k];
37.         }
38.         Rou = sqrt(SQR(dPos[0]) + SQR(dPos[1]) + SQR(dPos[2]));
39.         B(n, 0) = dPos[0] / Rou;
40.         B(n, 1) = dPos[1] / Rou;
41.         B(n, 2) = dPos[2] / Rou;
42.         w(n) = obs->CombObs[j].PIF - (Rou - obs->SatPVT[j].SatClkOft* CLIGHT + obs->SatPVT[j].TropCorr);
43.         if (obs->SatObs[j].System == GPS) {
44.             B(n, 3) = 1.0;
45.             B(n, 4) = 0.0;
46.             w(n) -= X0(3);
47.             GPSNum++;
48.         }
49.         else if (obs->SatObs[j].System == BDS) {
50.             B(n, 3) = 0.0;
51.             B(n, 4) = 1.0;
52.             w(n) = w(n) - X0(4) ;
53.             BDSNum++;
54.         }
55.         else continue;
56.         n++;
57.     }
58.     int dim = 5;
59.     if (n < 4) return false;
60.     BTB = B.block(0, 0, n, 5).transpose() * B.block(0, 0, n, 5);
61.     BTW = B.block(0, 0, n, 5).transpose() * w.head(n);
62.     if (GPSNum == 0) {
63.         for (int k = 0; k < 5; k++) {
64.             BTB(k, 3) = BTB(k, 4);
65.         }
66.         for (int k = 0; k < 5; k++) {
```

```
67.             BTB(3, k) = BTB(4, k);
68.         }
69.         BTW(3, 0) = BTW(4, 0);
70.         dim--;
71.         for (int k = 0; k < n; k++) {
72.             B(k, 3) = B(k, 4);
73.         }
74.     }
75.     else if (BDSNum == 0) {
76.         dim--;
77.     }
78.     else;
79.     X = BTB.block(0, 0, dim, dim).inverse() * BTW.block(0, 0, dim, 1);
80.     if (X.norm() < 1e-4) break;
81.     //更新用户位置
82.     for (int k = 0; k < 3; k++) {
83.         X0(k) += X(k);
84.     }
85.     UserPos->SigmaPos    = UserPos->PDOP = 0;
86.     V = B.block(0, 0, n, dim) * X.head(dim) - w.head(n);
87.     for (int k = 0; k < n; k++) {
88.         sigma2 += V(k) * V(k);
89.     }
90.     if (n <= dim) {
91.         UserPos->SigmaPos = 0.0;
92.     }
93.     else {
94.         UserPos->SigmaPos = sqrt(sigma2 / (n - dim));
95.     }
96.     MatrixXd Q = BTB.block(0, 0, dim, dim).inverse();
97.     UserPos->PDOP = sqrt(BTB.block(0, 0, dim, dim).inverse()(0, 0) +
98.         BTB.block(0, 0, dim, dim).inverse()(1, 1) +
99.         BTB.block(0, 0, dim, dim).inverse()(2, 2));
100.    //更新接收机钟差
101.    if (dim == 5) {
102.        X0(3) += X(3);
103.        X0(4) += X(4);
104.    }
105.    else if (dim == 4) {
106.        if (GPSNum == 0) {
107.            X0(4) += X(3);
108.        }
```

```
109.         else if (BDSNum == 0) {
110.             X0(3) += X(3);
111.         }
112.     }
113.     BTB.setZero();
114.     BTW.setZero();
115.     w.setZero();
116.     V.setZero();
117. }
118. UserPos->Pos[0] = X0(0);
119. UserPos->Pos[1] = X0(1);
120. UserPos->Pos[2] = X0(2);
121. UserPos->RcvClk[0] = X0(3) / CLIGHT;
122. UserPos->RcvClk[1] = X0(4) / CLIGHT;
123. UserPos->SatNum = n;
124. return true;
125. }
```

2.5.3 单点测速

2.5.3.1 计算公式

对于第j颗卫星：

$$D_j = \dot{\rho}_j + c \cdot \delta\dot{t}_j + \varepsilon_j \tag{135}$$

其中D_j是观测多普勒频移；$\dot{\rho}_j$是卫星与接收机之间的相对径向速度；$\delta\dot{t}_j$是卫星钟漂；c是光速；ε_j是观测误差。

接着计算相对速度：

$$\rho_j = \|r_R - r_j\| \tag{136}$$

$$\dot{\rho}_j = \frac{(r_R - r_j)^T v_j}{\rho_j} \tag{137}$$

其中ρ_j是空间距离；$\dot{\rho}_j$是相对速度投影；$r_R = (x, y, z)$是接收机的已知位置。

接着构建现行观测模型：

$$x = [V_x, V_y, V_z, c \cdot \delta \dot{t}_R]^T \tag{138}$$

$$w_j = D_j - (\dot{\rho}_j - c \cdot \delta \dot{t}_j) \tag{139}$$

$$B_j = \left[\frac{x_R - x_j}{\rho_j} \quad \frac{y_R - y_j}{\rho_j} \quad \frac{z_R - z_j}{\rho_j} \quad 1\right] \tag{140}$$

其中x为状态变量；w_j为每颗卫星的线性方程；B_j为对应观测矩阵的第j行内容。

接着进行最小二乘解算：

$$w = B \cdot x + v \tag{141}$$

$$N = B^T B, W = B^T w \tag{142}$$

$$\hat{x} = N^{-1} \cdot W \tag{143}$$

$$\hat{v}_R = \begin{bmatrix} V_x \\ V_y \\ V_z \end{bmatrix} = \begin{bmatrix} \hat{x}_{(1)} \\ \hat{x}_{(2)} \\ \hat{x}_{(3)} \end{bmatrix}, c \cdot \delta \dot{t}_R = x_{(4)} \tag{144}$$

其中w是残差向量；N,W是法方程；$\hat{x}$是最小二乘解；$\hat{v}_R$是解算速度。

最后计算速度估计的精度，即单位权中误差：

$$\sigma_v = \sqrt{\frac{v^T \cdot v}{n-4}} = \sqrt{\frac{\sum_j (w_j - \hat{w}_j)^2}{n-4}} \tag{145}$$

其中σ_v是速度的单位权中误差。

2.5.3.2 程序实现

```
void SPV(EPOCHOBSDATA* obs, SPPPOS* UserPos){
    int n;
    double dPos[3],Rou,Rou_dot;
    UserPos->Time = obs->Time;
    n = obs->SatNum;
    MatrixXd B(n, 4), BTPB(4, 4), BTPW(4, 1), Q(4, 4);
    VectorXd w(n), X(4),v(n);
    B.setZero();
    w.setZero();
    int m = 0;
    for (int i = 0; i < obs->SatNum; i++) {
        if (obs->SatPVT[i].Valid == false || obs->SatObs[i].valid
    == false) {
            continue;
        }
        // 计算接收机到卫星的位置
        dPos[0] = UserPos->Pos[0] - obs->SatPVT[i].SatPos[0];
```

```
        dPos[1] = UserPos->Pos[1] - obs->SatPVT[i].SatPos[1];
        dPos[2] = UserPos->Pos[2] - obs->SatPVT[i].SatPos[2];
        Rou = sqrt(SQR(dPos[0]) + SQR(dPos[1]) + SQR(dPos[2]));
        // 计算卫星的相对速度 Rou_dot
        Rou_dot = -(dPos[0] * obs->SatPVT[i].SatVel[0] +
            dPos[1] * obs->SatPVT[i].SatVel[1] +
            dPos[2] * obs->SatPVT[i].SatVel[2]) / Rou;
        //构造观测矩阵
        B(m, 0) = dPos[0] / Rou;
        B(m, 1) = dPos[1] / Rou;
        B(m, 2) = dPos[2] / Rou;
        B(m, 3) = 1.0;
        // 计算残差
        w(m) = obs->SatObs[i].D[0] - (Rou_dot - CLIGHT * obs->SatPVT[i].SatClkSft);
        m++;
    }
    if (m < 4) return;
    BTPB = B.block(0,0,m,4).transpose() * B.block(0, 0, m, 4);
    BTPW = B.block(0, 0, m, 4).transpose() * w.head(m);
    X = BTPB.inverse() * BTPW;
    v = B.block(0, 0, m, 4) * X - w.head(m);
    UserPos->SigmaVel = sqrt((v.transpose() * v)(0, 0) / (m - 4));
    Q = BTPB.inverse();
    UserPos->Dxx = UserPos->SigmaVel * Q;
    UserPos->Vel[0] = X(0);
    UserPos->Vel[1] = X(1);
    UserPos->Vel[2] = X(2);
}
```

2.6 获取实时数据流

```
#include<stdio.h>
#include<windows.h>
#pragma comment(lib,"WS2_32.lib")
#pragma warning(disable:4996)
bool OpenSocket(SOCKET& sock, const char IP[], const unsigned short Port){
    WSADATA wsaData;
    SOCKADDR_IN addrSrv;
    if(!WSAStartup(MAKEWORD(1, 1), &wsaData)){
        if( (sock = socket(AF_INET, SOCK_STREAM ,0)) != INVALID_SOCKET ){
            addrSrv.sin_addr.S_un.S_addr = inet_addr(IP);
            addrSrv.sin_family = AF_INET;
            addrSrv.sin_port = htons(Port);
            connect(sock, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));
            return true;
        }
    }
    return false;
}
void CloseSocket(SOCKET& sock){
    closesocket(sock);
    WSACleanup();
```

```
22. }
```

2.7 主函数设计

2.7.1 实现逻辑

本程序的主函数是整个导航解算软件的运行入口，主要负责完成数据读取、数据解码、粗差探测、卫星位置计算、SPP 定位、SPV 测速以及结果输出等流程。程序支持两种运行模式：文件模式用于读取本地 OEM719 二进制数据，网络模式用于实时接收 GNSS 数据流。主函数具体流程如下：

2.7.1.1 初始化数据结构

程序首先定义文件指针、网络套接字、数据缓冲区以及读取长度等变量，并初始化观测数据结构Obs、GPS 星历数组$GpsEph$、BDS 星历数组$BdsEph$和定位结果结构体$UserPos$。同时设置总历元数和 SPP 成功历元数统计变量。

2.7.1.2 选择数据输入方式

程序根据$FILEMODE$判断数据来源。若$FILEMODE == 1$，则打开本地 OEM719 原始数据文件，并创建日志文件$oem.log$；否则调用$OpenSocket()$函数，通过指定 IP 地址和端口号建立网络连接，实时接收 GNSS 数据。

2.7.1.3 循环读取原始数据

程序进入主循环后，根据不同模式读取数据。文件模式下通过$fread()$读取本地二进制数据；网络模式下通过$recv()$接收实时数据流。读取到的数据被连续存入缓冲区$Buff$，等待后续解码。

2.7.1.4 解码 OEM7 数据

程序调用$DecodeNovOem7Dat()$对缓冲区中的原始数据进行解析，提取当前历元观测值、GPS 星历和 BDS 星历。当函数返回值为 1 时，表示成功获得一个完整历元，程序继续进行后续解算。

2.7.1.5 粗差探测

获得完整历元后，程序调用$DetectOutlier()$对观测数据进行粗差探测，剔除或标记异常观测值。若当前为第一个历元，则设置接收机三维坐标和 GPS/BDS 接收机钟差初值，为后续 SPP 迭代解算提供初始条件。

2.7.1.6 卫星 PVT 计算

程序调用$SatPVTCalculation()$，利用广播星历和观测数据计算各卫星的位置、速度、钟差、钟漂、高度角和对流层改正等信息。该部分结果是后续 SPP 定位和 SPV 测速的基础。

2.7.1.7 SPP 定位与 SPV 测速

程序首先调用$SPP()$进行单点定位解算，估计接收机坐标和钟差。若 SPP 解算成功，则继续调用$SPV()$进行单点测速，计算接收机三维速度；若 SPP 失败，则输出失败信息并跳过当前历元。

2.7.1.8 结果输出与资源释放

定位成功后，程序输出当前历元的时间、坐标、速度、接收机钟差、PDOP、定位精度以及 GPS/BDS 卫星数等信息。程序结束时，关闭文件或网络连接，并输出总历元数和 SPP 成功数。

2.7.2 C++实现

2.7.2.1 初始化变量和结构体

```
1.  FILE* fp = nullptr;
2.  SOCKET NetGPS;
3.  unsigned char Buff[MAXRAWLEN];
4.  int Len = 0;
5.  int LenRead = 0;
6.  int val = 0;
7.  EPOCHOBSDATA Obs;
8.  GPSEPH GpsEph[MAXGPSNUM];
9.  GPSEPH BdsEph[MAXBDSNUM];
10. SPPPOS UserPos;
11. memset(GpsEph, 0, sizeof(GpsEph));
12. memset(BdsEph, 0, sizeof(BdsEph));
```

```
13.  memset(&UserPos, 0, sizeof(UserPos));
```

2.7.2.2 选择文件模式或网络模式

```
1.  if (FILEMODE == 1){
2.      flog.open("oem.log", ios::out);
3.      fp = fopen("oem719-202603111200.bin", "rb");
4.  }
5.  else{
6.      OpenSocket(NetGPS, "47.114.134.129", 7190);
7.  }
```

2.7.2.3 循环读取数据

```
1.  do{
2.      if (FILEMODE == 1) {
3.          LenRead = fread(Buff + Len, sizeof(unsigned char), MAXRAWLEN - Len, fp);
4.          Len += LenRead;
5.      }
6.      else{
7.          Sleep(980);
8.          LenRead = recv(NetGPS, (char*)Buff + Len, MAXRAWLEN - Len, 0);
9.          Len += LenRead;
10.     }
```

2.7.2.4 数据解码与粗差探测

```
1.  val = DecodeNovOem7Dat(Buff, Len, &Obs, GpsEph, BdsEph);
2.  if (val == 1){
3.      totalEpochCount++;
4.      DetectOutlier(&Obs);
5.      if (totalEpochCount == 1) {
6.          UserPos.Pos[0] = 1000.0;
7.          UserPos.Pos[1] = 1000.0;
8.          UserPos.Pos[2] = 1000.0;
9.          UserPos.RcvClk[0] = 0.0;
10.         UserPos.RcvClk[1] = 0.0;
11.     }
```

2.7.2.5 卫星位置 PVT 计算与 SPP/SPV 解算

```
1.  SatPVTCalculation(GpsEph, BdsEph, &Obs, &UserPos);
2.  UserPos.Time = Obs.Time;
3.  if (SPP(GpsEph, BdsEph, &UserPos, &Obs)){
```

```
        successfulSPPCount++;
        SPV(&Obs, &UserPos);
        double blh[3] = { 0.0, 0.0, 0.0 };
        XYZToBLH(UserPos.Pos, blh, R_WGS84, F_WGS84);
    }
    else{
        cout << "SPP FAILED" << endl;
    }
```

2.7.2.6 结果输出并释放资源

```
    cout << "SPP: "
         << UserPos.Time.Week << " "
         << UserPos.Time.SecOfWeek
         << " X:" << UserPos.Pos[0]
         << " Y:" << UserPos.Pos[1]
         << " Z:" << UserPos.Pos[2]
         << " Vx:" << UserPos.Vel[0]
         << " Vy:" << UserPos.Vel[1]
         << " Vz:" << UserPos.Vel[2]
         << " PDOP:" << UserPos.PDOP
         << endl;
    } while (1);
    if (FILEMODE == 1){
        fclose(fp);
        flog.close();
    }
    else{
        CloseSocket(NetGPS);
    }
```

3 结果精度分析

3.1 数据与策略说明

本次评估使用的是 2026 年 6 月 6 日 15：00 至 2026 年 6 月 7 日 15：00 共 24 小时的实时数据。采用的软件是 RTKNav.cpp 进行单点定位与测速。

3.2 结果说明

进行定位计算得到接收机三维坐标位置后，我们使用解算结果与平均值进行对比分析，计算得到 ENU 三个方向的三维误差，并将结果保存在 result.txt 中，如图 7 所示，格式如表 4 所示：

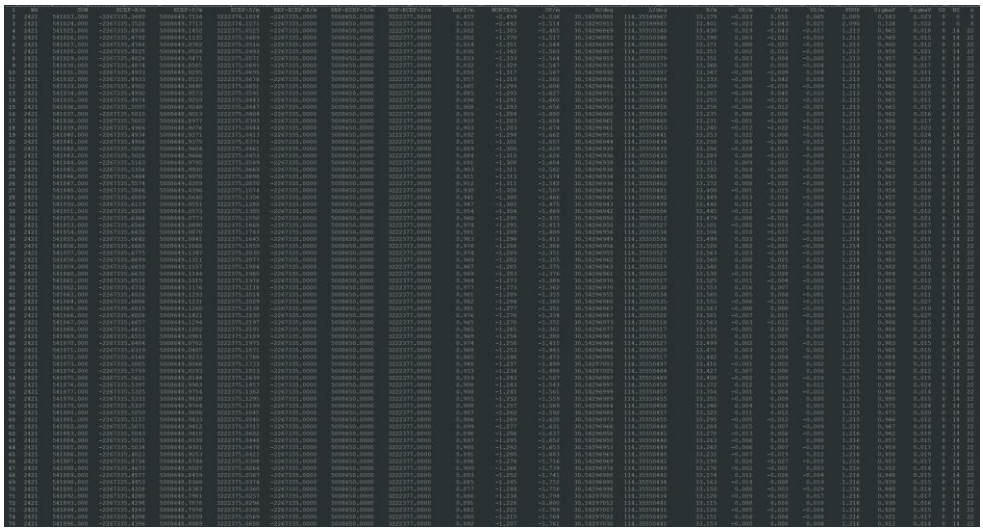


图 7 result.txt 格式示意图

表 4 定位结果格式说明表

名称	描述	单位
Week	周	/
SOW（Second Of Week）	周内秒	秒
ECEF-X/Y/Z	笛卡尔坐标系位置	米
REF-ECEF-X/Y/Z	参考坐标的地心地固坐标系下的坐标	米
EAST/NORTH/UP	东/北/天方向误差	米
B/L	纬度/经度	度
H	大地高	米
VX/VY/VZ	X/Y/Z 方向的速度	米每秒
PDOP	位置精度衰减因子	/
SigmaP	位置标准差	米
SigmaV	速度标准差	米每秒
BS	BDS 卫星数	个
GS	GPS 卫星数	个
n	总卫星数	个

3.3 计算精度分析

本次实验采用实时模式对 GPS/BDS 双系统观测数据进行单点定位与测速解算，并分别绘制了定位误差、速度误差、可见卫星数量和 PDOP 变化结果。整体来看，程序能够连续完成 24 h 实时解算，定位结果未出现长时间发散，速度结果整体较稳定，说明实时数据接收、星历解码、卫星位置计算、误差改正和最小二乘估计等主要模块均能够正常工作。

从结果上看，定位误差主要表现为米级波动，速度误差主要表现为厘米每秒至分米每秒量级波动。定位误差大于速度误差的原因在于，单点定位主要依赖伪距观测，而伪距观测容易受到卫星轨道钟差、大气延迟、多路径效应和接收机噪声等因素影响；测速主要依赖多普勒观测，其短时间随机噪声通常小于伪距观测，因此速度解更加平稳。GNSS 误差源通常包括卫星轨道和钟差误差、大气传播延迟、多路径效应、接收机噪声等，这些因素共同决定了单点定位的误差水平。

3.3.1 定位误差精度分析

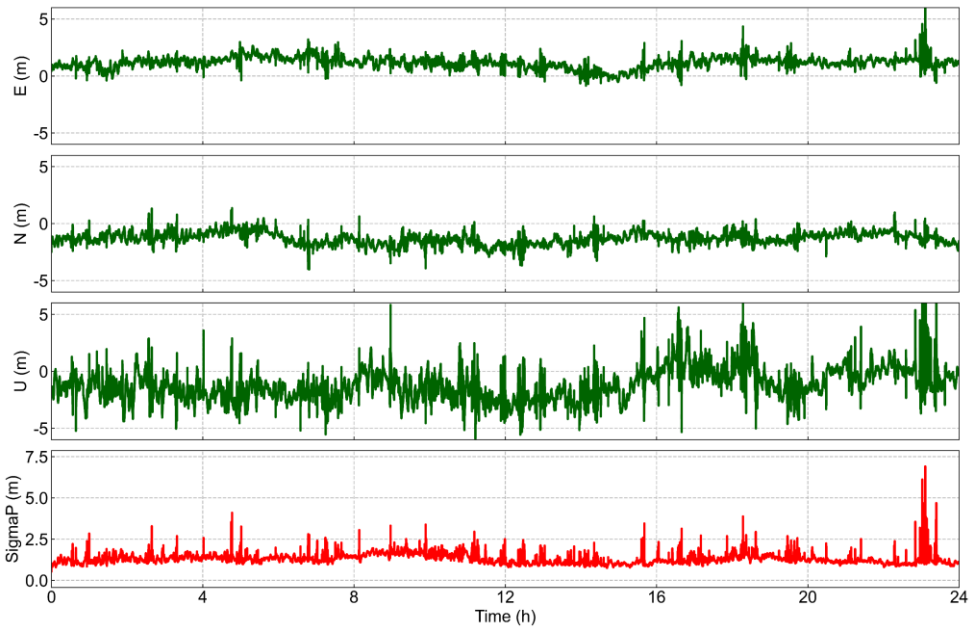


图 8 ENU 定位误差及标准差时间序列图

由图 8 可以看出，本次实时单点定位结果在 24 h 内整体保持连续稳定，E、N、U 三个方向的定位误差均未出现长时间发散现象，说明程序能够较稳定地完成 GPS/BDS 双系统实时单点

定位解算。其中，E 方向误差主要集中在 0 m 附近，整体波动幅度较小，但在 23 h 附近出现短时尖峰；N 方向误差整体存在一定负偏，说明北向分量中仍可能存在未完全消除的系统性残差；U 方向误差波动最为明显，部分时段可达到数米量级，尤其在 16 h 以后误差起伏明显增大。整体来看，垂直方向误差明显大于水平分量，这是 GNSS 单点定位中的典型现象。

造成 U 方向误差较大的主要原因是卫星几何构型对垂直方向约束较弱。对于地面接收机而言，所有可见卫星均位于测站上方半球空间，地平线以下不存在可用卫星信号源。因此，卫星观测对水平方向的约束通常强于垂直方向，而垂直方向的定位结果更容易受到观测误差放大影响。即使本次实验中可见卫星数量较多，仍无法完全弥补垂直方向几何构型上的天然不足，因此 U 方向误差表现出更大的波动幅度。

此外，大气延迟残差也是导致垂直方向误差较大的重要因素。GNSS 信号传播过程中会受到电离层和对流层影响。双频组合可以削弱一阶电离层延迟，但对流层延迟仍需要依靠模型进行改正。由于对流层延迟与卫星高度角密切相关，低高度角卫星信号传播路径更长，其残余误差更大，并且更容易投影到垂直方向。因此，当某些时段低高度角卫星参与解算，或者对流层模型与实际大气状态存在偏差时，U 方向误差会明显增大。

图 8 中的 SigmaP 反映了定位结果的整体精度估计。可以看出，SigmaP 大部分时间保持在较低水平，但在 23 h 附近出现明显尖峰，与 E、U 方向误差突变具有一定对应关系。这说明该时段定位解的不确定性增大，可能与卫星数量变化、卫星几何结构变化、观测质量下降或伪距粗差有关。因此，在分析定位精度时，不应仅关注 E、N、U 误差本身，还应结合 SigmaP 判断定位解的可靠性。

3.3.2 单点测速精度分析

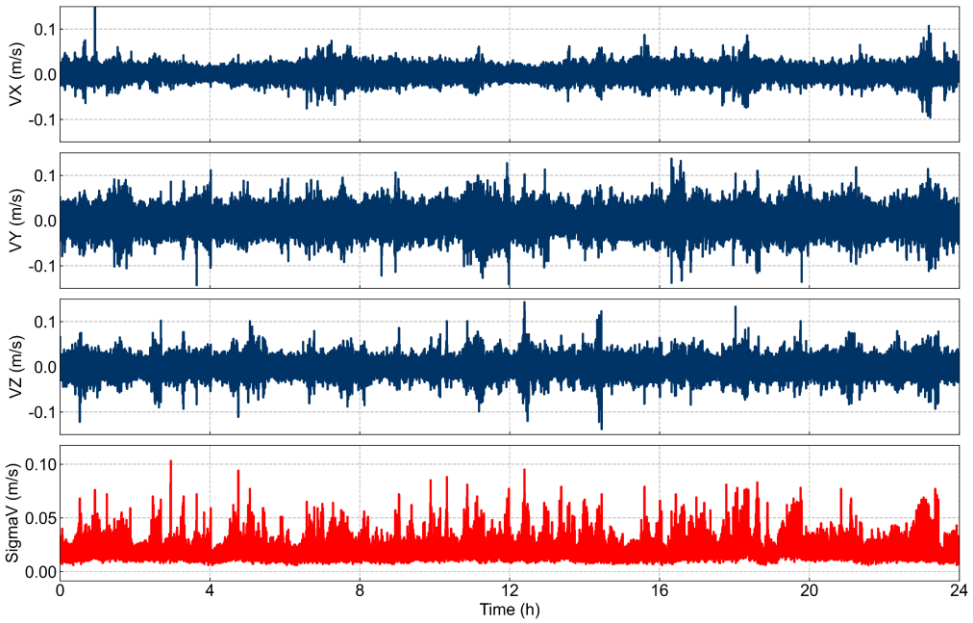


图 9 速度解算结果及其标准差时间序列图

由图 9 可以看出，VX、VY、VZ 三个方向的速度较为平稳，说明本次程序的实时测速结果较为稳定。与伪距定位不同，GNSS 测速主要依赖多普勒观测。多普勒观测反映的是卫星与接收机之间距离变化率，其短时间随机噪声通常小于伪距观测，因此速度解一般比位置解更加平稳。

从三个方向的速度误差来看，VX 方向整体波动较小，VY 和 VZ 方向的局部尖峰相对更多，说明不同方向的速度估计同样受到卫星几何结构和观测质量的影响。图中在若干时段出现瞬时尖峰，可能是由于卫星升落、信号短时失锁、多普勒观测粗差、接收机钟漂估计误差或低高度角卫星观测质量下降造成的。由于实时解算是逐历元进行的，当某一历元的多普勒观测出现异常时，速度解容易表现为短时跳变。

图 9 中的 SigmaV 整体保持在较低水平，多数时段约为 0.01 m/s 至 0.03 m/s，说明速度解的随机精度较好。但局部时段 SigmaV 也出现明显增大，最高接近 0.1 m/s，表明该时段测速结果可靠性下降。结合速度误差曲线可知，SigmaV 尖峰通常与速度误差突变相对应，说明标准差指标能够较好反映实时测速过程中的异常历元。

总体而言，图 9 表明本次程序实现的单点测速性能整体稳定，未出现持续发散现象。这与 GNSS 测速理论相符，即多普勒观测受伪距多路径和大气延迟的影响相对较小，因此在短时间尺度上具有更高稳定性。

3.3.3 可见卫星数量变化分析

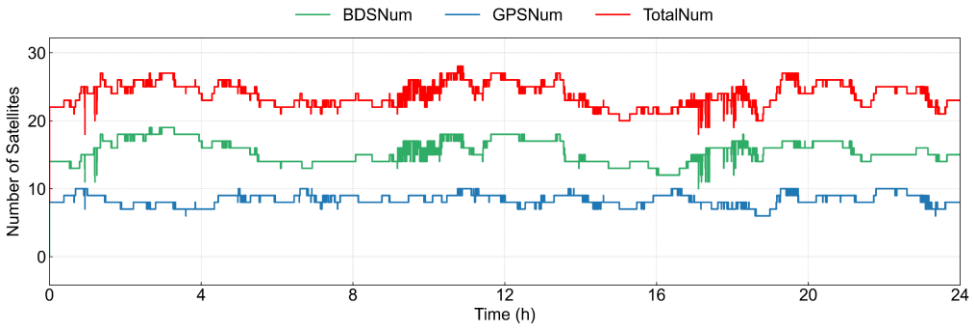


图 10 参与解算的卫星数量时间序列图

由图 10 可以看出，本次实验中 GPS/BDS 双系统可见卫星数量较为充足。GPS 卫星数量大致保持在 7 至 10 颗之间，BDS 卫星数量大致保持在 12 至 19 颗之间，总参与解算卫星数量大多保持在 20 颗以上，部分时段接近 30 颗。该结果说明实验时段内观测条件整体较好，能够满足实时单点定位和测速解算的基本需求。

从系统组成来看，BDS 卫星数量明显多于 GPS 卫星数量，说明 BDS 在本次双系统解算中对提高观测冗余度具有重要作用。多系统联合解算可以增加观测方程数量，提高最小二乘估计的冗余度，并改善卫星空间分布，从而增强定位解和速度解的稳定性。相比单 GPS 系统，GPS/BDS 双系统能够在卫星遮挡、卫星升落或部分卫星观测质量下降时提供更多可用观测，有利于提高实时解算的连续性。

但是，卫星数量多并不代表定位精度一定高。定位精度不仅取决于卫星数量，还与卫星空间分布、高度角、信号质量和观测误差水平有关。如果新增卫星主要集中在相近方向，或者多为低高度角卫星，其对几何结构的改善有限，甚至可能引入更大的大气延迟残差和多路径误差。因此，图 10 需要与图 11 中的 PDOP 变化以及图 8 中的定位误差共同分析。

从时间变化上看，图 10 中总卫星数量在部分时段存在小幅波动。例如在 16 h 至 18 h 附近，BDS 卫星数量出现一定变化，总卫星数量也随之波动。对应图 8 和图 11 可以发现，该时

段定位误差、SigmaP 和 PDOP 均有一定变化，说明卫星数量变化可能是影响定位结果稳定性的因素之一。当参与解算的卫星发生切换时，观测方程结构和几何构型也会改变，从而导致定位误差出现短时波动。

3.3.4 PDOP 变化分析

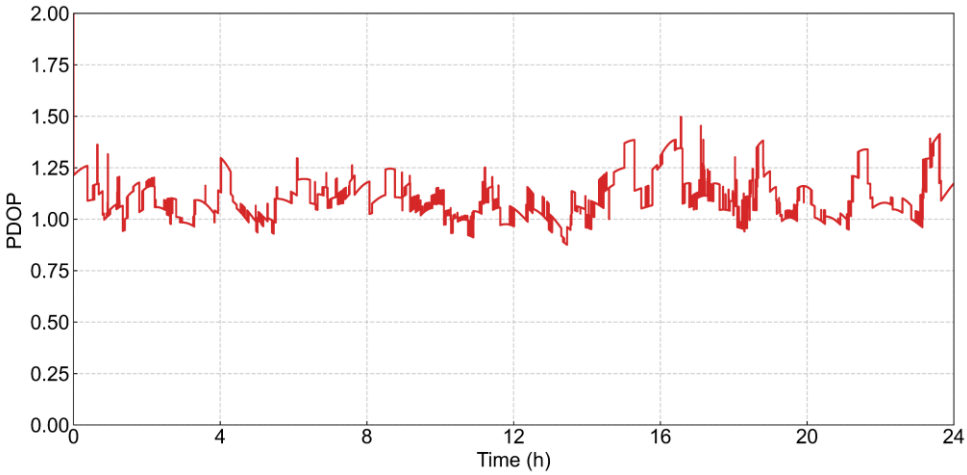


图 11 PDOP 值时间序列图

由图 11 可以看出，本次实验 24 h 内 PDOP 整体保持在较低水平，大部分时间位于 1.0 至 1.4 之间，说明 GPS/BDS 双系统的卫星几何构型较好，能够为实时单点定位提供较强的几何约束。PDOP 没有出现长时间大幅升高，表明整个观测时段的几何条件总体稳定。

PDOP 是位置精度衰减因子，用于描述卫星几何构型对定位误差的放大作用。在观测噪声相同的情况下，PDOP 越小，定位误差越不容易被放大；PDOP 越大，同样大小的伪距误差会造成更大的坐标误差。因此，PDOP 是评价定位解几何可靠性的重要指标。本次实验中 PDOP 整体较低，与图 10 中可见卫星数量较多的现象基本一致，说明双系统联合定位有效改善了卫星几何结构。

不过，图 11 中约 16 h 附近和 23 h 附近 PDOP 出现一定上升，其中 23 h 附近定位误差和 SigmaP 也出现明显增大。这说明当卫星空间分布变差或部分卫星退出解算时，定位结果会受到影响。尤其是在 23 h 附近，图 8 中 E 方向、U 方向和 SigmaP 均出现尖峰，说明几何结构变化可能与观测质量下降共同导致了该时段的定位异常。

需要注意的是，PDOP 只能反映卫星几何构型，不能完全代表观测质量。即使 PDOP 较小，如果观测数据中存在多路径误差、低高度角卫星误差、大气延迟残差或接收机噪声，定位结果仍可能出现较大波动。因此，PDOP 应与卫星数量、定位误差、速度误差以及标准差指标联合分析，才能更全面地评价实时解算结果。

3.3.5 误差主要影响因素分析

综合图 8 至图 11 可以看出，本次实时单点定位与测速误差主要受以下因素影响。

（1） 卫星几何构型

虽然本次实验中可见卫星数量较多，PDOP 整体较低，但由于地面接收机只能接收上方半球的卫星信号，垂直方向几何约束天然弱于水平方向。因此，图 8 中 U 方向误差明显大于 E、N 方向误差，这与 GNSS 定位理论一致。

（2） 大气延迟残差

双频组合可以有效削弱一阶电离层延迟，但对流层延迟仍依赖经验模型改正。在没有实时气象参数或对流层湿延迟精确估计的情况下，对流层模型残差难以完全消除。该误差在低高度角卫星上更加明显，并容易投影到垂直方向，是 U 方向误差较大的重要原因之一。

（3） 多路径效应

当 GNSS 信号受到地面、建筑物或其他障碍物反射时，接收机会同时接收到直接信号和反射信号，导致伪距观测产生偏差。该类误差具有较强的环境相关性，难以通过简单模型完全改正，因此容易在定位误差曲线中表现为短时异常。

（4） 接收机噪声

任何电子设备都存在内部噪声。GNSS 接收机在信号处理过程中引入的噪声通常会造成约 0.3 米至 1.0 米的测距误差。这种噪声是系统固有的，表现为误差时间序列图上高频的、毛刺状的波动，构成了定位结果的最终噪声基底。

（5） 卫星轨道和钟差误差

本次实时单点定位主要依赖广播星历计算卫星位置和钟差，而广播星历提供的卫星轨道和钟差信息存在一定精度限制。残余轨道钟差误差会直接进入伪距观测方程，并最终影响接收机坐标估计。

（6） 实时处理中的卫星切换和粗差探测能力

图 11 显示可见卫星数量随时间发生变化，当参与解算的卫星集合发生变化时，观测方程结构、冗余度和几何构型都会发生改变。如果此时存在伪距或多普勒异常观测，而粗差探测未能完全识别，就可能导致图 8 和图 9 中的局部尖峰。

总体而言，本次实验结果符合 GNSS 实时单点定位与测速的基本规律。定位结果整体达到米级精度，速度结果达到分米每秒以内的精度水平；GPS/BDS 双系统提供了较充足的卫星数量和较好的 PDOP 条件，保证了解算连续性和稳定性；局部时段误差增大主要由卫星几何变化、低高度角卫星观测质量下降、大气延迟残差、多路径效应以及实时粗差处理不足共同造成。